

Dockerfiles, Compose & Kubernetes Manifests Explained

All the code explained here is in the GitHub repository:
github.com/nitin1094/two-tier-flask-app-k8s

A Beginner's Line-by-Line Walkthrough — every Dockerfile, the Compose file, the Jenkins pipeline, and every manifest in `k8s/` and `eks-manifests/`, one line at a time.

What this guide covers

Dockerfile	Eleven lines that package the Flask app — and why their order matters
Dockerfile-multistage	The slim production image, and the one line that makes it work
docker-compose.yml	Both tiers on one machine: networks, volumes, health checks
Makefile & Jenkinsfile	Compose shortcuts, and a six-stage CI/CD pipeline explained
k8s/ manifests	PV & PVC, two Deployments, a bare Pod for contrast, a NodePort Service
eks-manifests/	A Secret, a ConfigMap that seeds the database, a LoadBalancer Service
Side by side	What actually changes between a self-managed cluster and Amazon EKS
Appendices	Field reference, error decoder, interview Q&A & glossary

Written in plain language for readers with only basic Docker and Kubernetes knowledge. Every file is quoted *verbatim* from the repository — nothing simplified, nothing invented — so you can open the real code alongside this guide and follow along. A green “From the repo:” link under each chapter heading points at the exact file.

Table of Contents

Every entry is a live link — click it to jump straight to that topic.

The App and the Two Kinds of Files	4
1. What This Guide Explains	4
2. Docker in Five Minutes	6
3. Kubernetes in Five Minutes	7
Building and Running It (the repo root)	10
4. The Dockerfile, Line by Line	10
5. Dockerfile-multistage - the Slim Version	14
6. docker-compose.yml - Both Tiers on One Machine	16
7. The Makefile - Shortcuts for Compose	20
8. The Jenkinsfile - Automating Build and Deploy	21
The k8s/ Folder (a self-managed cluster)	25
9. How to Read the k8s/ Manifests	25
10. Storage - mysql-pv.yml and mysql-pvc.yml	26
11. mysql-deployment.yml	28
12. mysql-svc.yml	32
13. two-tier-app-pod.yml - and Why You Would Not Use It	34
14. two-tier-app-deployment.yml	35
15. two-tier-app-svc.yml - the NodePort	38
The eks-manifests/ Folder (Amazon EKS)	39
16. What Changes on Amazon EKS	39
17. mysql-secrets.yml	41
18. mysql-configmap.yml	42
19. mysql-deployment.yml and mysql-svc.yml	43
20. two-tier-app-deployment.yml and the LoadBalancer	47

21. The Two Manifest Folders Side by Side	49
22. Field Reference Cheat Sheet	50
23. Common Errors and What They Mean	53
24. Interview Questions and Answers	54
25. Glossary	57
26. Links and Repository	58

The App and the Two Kinds of Files

What this guide covers, the Flask + MySQL app behind the files, and just enough Docker and Kubernetes to make every line that follows meaningful.

1. What This Guide Explains

This is a **code-reading guide**, not a deployment tutorial. Its job is to open every build file and every Kubernetes manifest in the two-tier Flask project and explain them **line by line**, in plain English, so nothing in them is a mystery to you.

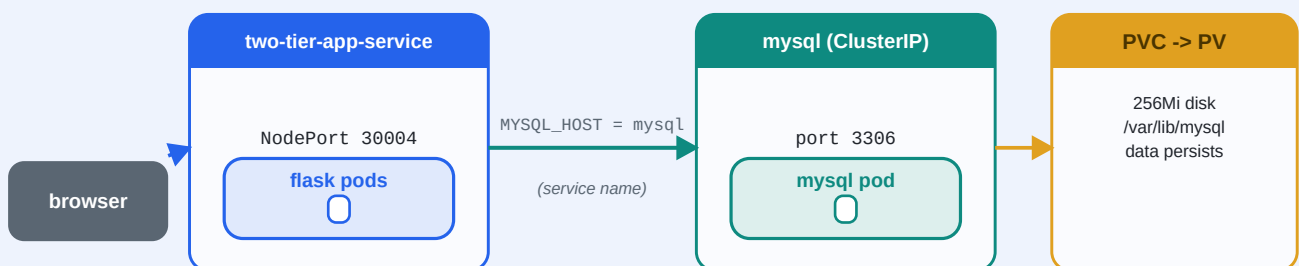
You need only basic Docker and Kubernetes knowledge: roughly what a container is, and that Kubernetes runs containers for you. Everything else is explained here as it comes up.

The app these files package

It is a small **message board**. You type a message, it is saved, and every message is shown on the page. Two tiers, hence the name:

Tier	What it is	Technology	Listens on
Application	The web page and the logic behind it	Python 3.9 + Flask	port 5000
Data	Where the messages are stored	MySQL	port 3306

The two-tier app running on Kubernetes



Flask pods are reached from outside via a NodePort Service. They find MySQL by its service NAME (service discovery), and MySQL keeps its data on a PersistentVolume.

The whole application is one file, [app.py](#), and it is worth seeing the four lines that matter most, because almost every file in this guide exists to satisfy them:

```

app.config['MYSQL_HOST'] = os.environ.get('MYSQL_HOST', 'localhost')
app.config['MYSQL_USER'] = os.environ.get('MYSQL_USER', 'default_user')
app.config['MYSQL_PASSWORD'] = os.environ.get('MYSQL_PASSWORD', 'default_password')
app.config['MYSQL_DB'] = os.environ.get('MYSQL_DB', 'default_db')
  
```

The app reads **where MySQL is** and **how to log in** from four environment variables. It hard-codes nothing. That single design choice is why the same image runs unchanged under Docker Compose, on a kubeadm cluster, and on Amazon EKS — only the four values change.

KEY IDEA Whenever you see `MYSQL_HOST`, `MYSQL_USER`, `MYSQL_PASSWORD` or `MYSQL_DB` in a compose file or a manifest in this guide, it is being fed to those four lines. Every file you are about to read is, in one way or another, a way of setting them.

Where the original code lives

Every file quoted here is copied **verbatim** from the project's GitHub repository. Nothing has been rewritten or tidied up — if a line looks odd, that is genuinely how it is, and this guide explains why.

From the repo: github.com/nitin1094/two-tier-flask-app-k8s

A green "**From the repo:**" link under each chapter heading points at the exact file, so you can open the real code beside this guide.

The files we will read

```
two-tier-flask-app-k8s/
+-- Dockerfile # Part II, chapter 4 - the everyday image
+-- Dockerfile-multistage # Part II, chapter 5 - the slim production image
+-- docker-compose.yml # Part II, chapter 6 - both tiers on one machine
+-- Makefile # Part II, chapter 7 - shortcuts for compose
+-- Jenkinsfile # Part II, chapter 8 - the CI/CD pipeline
+-- app.py, requirements.txt,
| message.sql, templates/ # the application itself
|
+-- k8s/ # PART III - a self-managed (kubeadm) cluster
| +-- mysql-pv.yml # chapter 10
| +-- mysql-pvc.yml # chapter 10
| +-- mysql-deployment.yml # chapter 11
| +-- mysql-svc.yml # chapter 12
| +-- two-tier-app-pod.yml # chapter 13
| +-- two-tier-app-deployment.yml # chapter 14
| +-- two-tier-app-svc.yml # chapter 15
|
+-- eks-manifests/ # PART IV - the same app on Amazon EKS
    +-- mysql-secrets.yml # chapter 17
    +-- mysql-configmap.yml # chapter 18
    +-- mysql-deployment.yml # chapter 19
    +-- mysql-svc.yml # chapter 19
    +-- two-tier-app-deployment.yml # chapter 20
    +-- two-tier-app-svc.yml # chapter 20
```

Why there are two manifest folders

This is the first thing that confuses people about this repo, so let us settle it now.

`k8s/` and `eks-manifests/` deploy **the same two-tier app**. They are not two halves of one deployment — they are **two complete alternatives**, and you pick one:

	<code>k8s/</code>	<code>eks-manifests/</code>
Built for	A cluster you run yourself, e.g. built with kubeadm	Amazon EKS, a managed cluster
Storage	A hostPath PersistentVolume + Claim	None — the database is not persisted
Credentials	Plain text in the Deployment	The root password moves into a Secret

	k8s/	eks-manifests/
Table creation	The app creates it at startup	A <code>ConfigMap</code> seeds it
Public access	<code>NodePort</code> — <code>http://<node-ip>:30004</code>	<code>LoadBalancer</code> — a real AWS load balancer

Part III reads the first folder, Part IV reads the second, and chapter 21 puts them side by side.

NOTE Do **not** apply both folders to the same cluster. They create objects with identical names (`mysql`, `two-tier-app`, `two-tier-app-service`), so the second `kubectl apply` would silently overwrite the first and you would end up with a confusing hybrid.

2. Docker in Five Minutes

Just enough Docker to make every line of Part II meaningful.

The three levels: Dockerfile, image, container

Think of cooking:

- A **Dockerfile** is the **recipe** — a plain text file of instructions, written once.
- An **image** is the **frozen meal** made by following that recipe — a read-only, shippable bundle containing your app, its libraries, and the operating-system files it needs.
- A **container** is the **meal being eaten** — a running copy of the image. Many containers can run from one image without interfering.

`docker build` turns the recipe into the frozen meal; `docker run` serves it. In Part III and IV, Kubernetes does the serving.

ANALOGY An image is like a class in programming; a container is like an object created from it. Write the class once, make as many objects as you like.

Images are built in layers

Every instruction (`FROM`, `COPY`, `RUN`, ...) creates one **layer** — a saved snapshot of the filesystem at that point. Layers stack to form the final image.

This matters for one practical reason: **Docker caches layers**. On a rebuild, Docker walks the file from the top and reuses each cached layer until it hits an instruction whose inputs changed. From there down, everything is rebuilt.

That single fact explains the shape of the Dockerfile in chapter 4.

The instructions used in this project

Instruction	What it does
FROM	Picks the image to build on top of. Always the first real instruction.
WORKDIR	Sets the "current folder" inside the image for everything after it. Creates it if missing.
COPY	Copies files from your machine (the <i>build context</i>) into the image.
RUN	Runs a command while building , saving the result as a layer.
CMD	The default command to run when a container starts — not during the build.
AS <name>	Names a build stage, so a later stage can copy from it (chapter 5).
COPY --from=	Copies files out of an earlier stage instead of from your machine (chapter 5).

NOTE `RUN` happens during `docker build`; `CMD` happens during `docker run`. Mixing these two up is the most common beginner confusion. The test: if it should be *baked into* the image, it is a `RUN`. If it should happen *fresh each time* a container starts, it is `CMD`.

3. Kubernetes in Five Minutes

Now the vocabulary for Parts III and IV.

Everything is a described object

You never tell Kubernetes *how* to do something. You hand it a YAML file describing **what you want**, and Kubernetes works continuously to make reality match. This is called being **declarative**, and the loop that keeps reality in sync is **reconciliation**.

Read every manifest as a wish, not a script: *"I want one MySQL pod, using this image, with this volume attached."*

The four fields every manifest starts with

Anatomy of a manifest (every Kubernetes object is a YAML file)



Every YAML file in `k8s/` and `eks-manifests/` opens with the same keys:

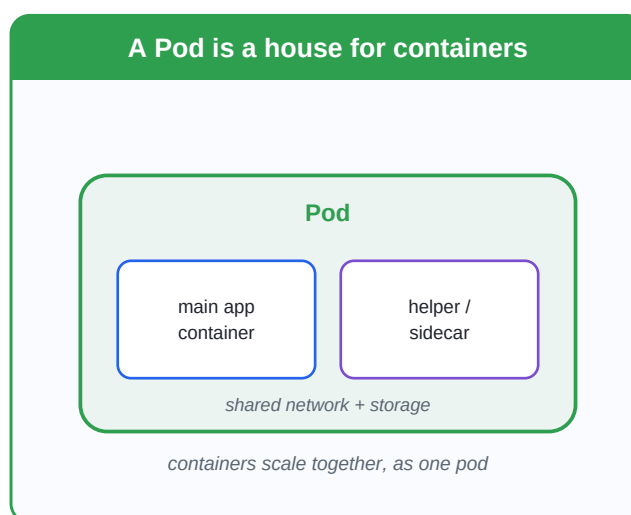
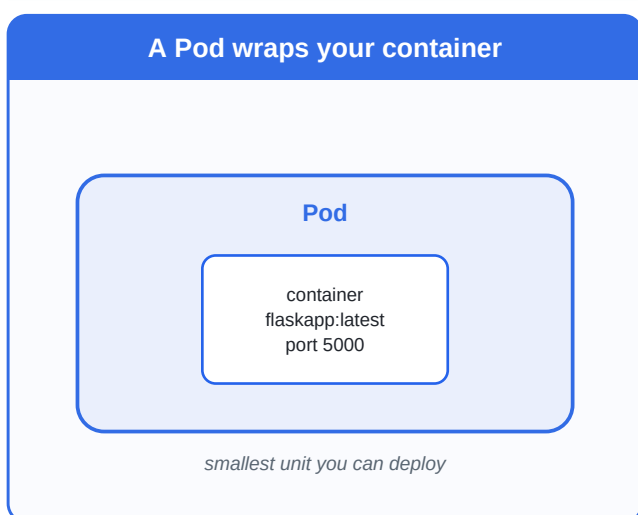
Field	Meaning	Examples in this project
<code>apiVersion</code>	Which version of the Kubernetes API this object belongs to	<code>v1</code> , <code>apps/v1</code>
<code>kind</code>	What type of object you are creating	<code>Deployment</code> , <code>Service</code> , <code>Pod</code> , <code>Secret</code> , <code>ConfigMap</code>
<code>metadata</code>	Its name and labels	<code>name: mysql</code> , <code>app: two-tier-app</code>
<code>spec</code>	The actual wish — everything specific to that kind	<code>replicas</code> , <code>image</code> , <code>ports</code> , <code>volumes</code> ...

Recognise this skeleton and every manifest becomes far less intimidating: the top lines tell you *what* you are looking at, and only `spec` really differs between files.

TIP `apiVersion: v1` covers the oldest, most fundamental objects (`Pod`, `Service`, `Secret`, `ConfigMap`, `PersistentVolume`, `PersistentVolumeClaim`). `apps/v1` covers workload controllers (`Deployment`). A "no matches for kind" error almost always means a wrong `apiVersion`.

The object types you will meet

Object	In one sentence
Pod	The smallest unit Kubernetes runs — one or more containers sharing a network address.
Deployment	A manager that keeps N identical pods alive and replaces them when they die.
Service	A stable name and IP in front of a changing set of pods.
Secret	A place to keep credentials out of your Deployment files.
ConfigMap	The same idea for <i>non</i> -secret configuration — settings, scripts, seed SQL.
PersistentVolume (PV)	Storage that exists independently of any pod, so data survives a restart.
PersistentVolumeClaim (PVC)	A pod's <i>request</i> for storage: "I need 256 MiB."



Labels and selectors — the glue

This one concept ties nearly every manifest in this repo together.

A **label** is a key/value sticker you put on an object:

```
labels:
  app: mysql # a sticker saying "this pod is the database"
```

A **selector** is a search for that sticker:

```
selector:
  app: mysql # "find me everything wearing the 'app: mysql' sticker"
```

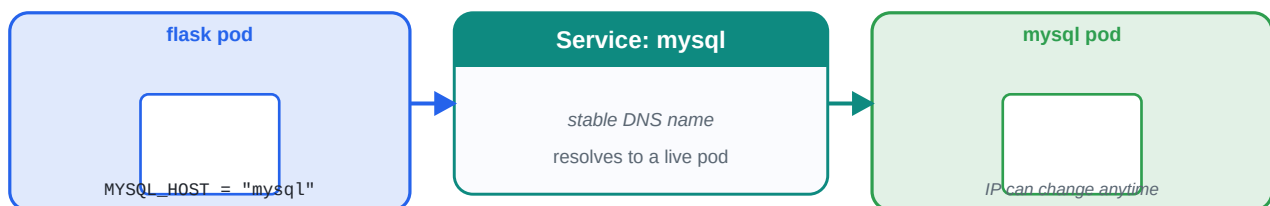
Deployments use a selector to know which pods they own. Services use one to know where to send traffic. Nothing is wired by IP address — **everything is wired by matching labels**.

Two label values do all the work here:

Label	Worn by	Searched for by
<code>app: two-tier-app</code>	the Flask pods	the <code>two-tier-app</code> Deployment and <code>two-tier-app-service</code>
<code>app: mysql</code>	the MySQL pod	the <code>mysql</code> Deployment and the <code>mysql</code> Service

WATCH OUT If a Service's selector does not exactly match the labels on the pods, the Service has **no endpoints** and traffic silently goes nowhere — with no error message. When something "just doesn't connect", check `kubectl get endpoints` before anything else.

Service discovery — the trick that makes two tiers work



Never hard-code a pod's IP. Point to the service NAME and Kubernetes routes to a healthy pod.

The Flask app is told `MYSQL_HOST=mysql`. That is not a hostname you configured anywhere — it is the **name of the MySQL Service**. Kubernetes runs an internal DNS server that turns every Service name into its virtual IP automatically.

So the app connects to `mysql:3306`, Kubernetes resolves `mysql` to the Service, and the Service forwards to whichever pod currently matches `app: mysql`. The pod can be deleted and recreated with a brand-new IP a hundred times; the name `mysql` never changes.

KEY IDEA Pods are disposable and their IPs change constantly. Services exist so that nothing in your application ever has to care. This is the single most important idea in Parts III and IV.

One thing that is missing everywhere: a namespace

A **namespace** is a folder inside the cluster that groups related objects. The three-tier project in the companion guide puts everything in a `three-tier` namespace.

This project uses none. No manifest in either folder has a `namespace:` field, so every object lands in the namespace called `default`.

That is why none of the `kubectl` commands in this guide carry a `-n` flag. It is fine for a learning project, but on a shared cluster it means your objects sit alongside everyone else's.

TIP To keep this project tidy anyway, create a namespace and apply into it without editing a single file: `kubectl create namespace two-tier` then `kubectl apply -f k8s/ -n two-tier`. The `-n` flag applies to every object that does not name its own namespace.

PART II

Building and Running It (the repo root)

Both Dockerfiles line by line, the Compose file that runs the two tiers together, the Makefile shortcuts, and the Jenkins pipeline.

4. The Dockerfile, Line by Line

From the repo: [Dockerfile](#)

This is the everyday image — the one the Jenkins pipeline builds and the one whose name appears in every manifest in Parts III and IV. Here is the whole file:

```
# Use an official Python runtime as the base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# install required packages for system
RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Copy the requirements file into the container
COPY requirements.txt .

# Install app dependencies
RUN pip install mysqlclient
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Specify the command to run your application
CMD ["python", "app.py"]
```

Line 1 — FROM python:3.9-slim

```
FROM python:3.9-slim # official Python 3.9 on a trimmed-down Debian base
```

Every image builds **on top of another image**, and `FROM` picks that starting point — the **base image**. `python:3.9-slim` is the official Python image, which already contains a Debian Linux filesystem, Python 3.9 and `pip`, so you inherit all of it for free.

The `-slim` suffix matters. The plain `python:3.9` image is about 900 MB because it ships compilers, headers and developer tooling. `-slim` strips those out and lands around 120 MB. That is why the very next instruction has to install `gcc` by hand — `-slim` removed it.

WATCH OUT Python 3.9 reached end of life in **October 2025**, so it no longer gets security patches. The project uses it because its dependencies (Flask 2.0.1, Werkzeug 2.2.2) were written for that era. For anything real, move to a supported release such as `python:3.12-slim` — you will need to update the pinned package versions in `requirements.txt` at the same time.

Line 2 — WORKDIR /app

```
WORKDIR /app # cd into /app inside the image; create it if it does not exist
```

`WORKDIR` is Docker's `cd`, with a bonus: it creates the folder if it is missing. From here on, every `COPY`, `RUN` and `CMD` is relative to `/app`, which is why the next instruction can write `COPY requirements.txt .` and mean `/app/requirements.txt`.

NOTE Always use `WORKDIR`, never `RUN cd /some/dir`. A `RUN cd` affects only that one instruction, because each instruction runs in its own fresh shell — the next one starts back where it was. `WORKDIR` persists for the rest of the build *and* for the running container.

Lines 3–6 — installing the system packages

```
RUN apt-get update \  
    && apt-get upgrade -y \  
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \  
    && rm -rf /var/lib/apt/lists/*
```

The backslashes join four shell commands into **one** `RUN`, and therefore **one layer**. That is deliberate and it is the single most important habit in Dockerfile writing. Split across four separate `RUN` instructions, Docker could reuse a stale `apt-get update` layer on a later build and then try to install packages from an index that no longer matches — the classic *"404 Not Found"* apt failure inside Docker.

Each piece does something specific:

Command	Why it is there
<code>apt-get update</code>	Refreshes the list of available packages. Nothing can install without it.
<code>apt-get upgrade -y</code>	Upgrades already-installed packages. <code>-y</code> answers "yes" so the build is not left waiting for input.
<code>gcc</code>	A C compiler. Needed because <code>mysqlclient</code> is not pure Python — it compiles C code at install time.
<code>default-libmysqlclient-dev</code>	The MySQL client headers and library that the C code compiles against.
<code>pkg-config</code>	A helper the build uses to locate those headers. Without it the install fails with a confusing error.
<code>rm -rf /var/lib/apt/lists/*</code>	Deletes the downloaded package index — roughly 40 MB of files the running app will never use.

That last line is worth dwelling on. Because it is inside the **same** `RUN`, the index files are deleted before the layer is saved, so they never enter the image at all. If you deleted them in a *separate* `RUN`, the files would already be baked into the earlier layer and the image would still carry them — layers are append-only, and deleting a file in a later layer only hides it.

KEY IDEA Anything you download and then delete must be deleted **in the same RUN** that downloaded it. Otherwise it stays in the image forever, invisible but still taking up space.

TIP `apt-get upgrade -y` inside a Dockerfile is generally discouraged. It makes builds non-reproducible — the same Dockerfile produces different images on different days — and it is slow. The usual advice is to leave it out and instead rebuild on a newer base image tag when you want security updates.

Lines 7–9 — dependencies before code

```
COPY requirements.txt . # copy ONLY the dependency list
RUN pip install mysqlclient # the MySQL driver (compiled with gcc, above)
RUN pip install --no-cache-dir -r requirements.txt # everything else
```

Only `requirements.txt` is copied at this point, on its own, before any application code. This is deliberate, and it is where the layer caching from chapter 2 pays off:

You changed...	COPY requirements.txt	the two pip install lines	COPY . .
Only <code>app.py</code> (typical)	cache hit	cache hit — skipped entirely	rebuilt (fast)
<code>requirements.txt</code> (added a library)	cache miss	re-run (slow, but necessary)	rebuilt

Because you edit code far more often than you add libraries, this ordering turns a two-minute rebuild into a two-second one. Had the file said `COPY . .` and then `pip install`, **every** code change would force a full reinstall — including recompiling `mysqlclient` with `gcc` each time.

`requirements.txt` itself is four pinned lines:

```
Flask==2.0.1
Flask-MySQLdb==0.2.0
requests==2.26.0
werkzeug==2.2.2
```

`mysqlclient` is installed separately on the line above because `Flask-MySQLdb` needs it but the pinned combination installs more reliably when it is done first.

`--no-cache-dir` tells pip not to keep its download cache inside the image — the same "don't ship what you don't need" idea as the `rm -rf` earlier, worth roughly another 20 MB.

TIP Pinning exact versions (`Flask==2.0.1`, not `Flask`) is good practice and this project gets it right. Without pins, rebuilding the same Dockerfile months later silently pulls newer libraries and your image can break for reasons that have nothing to do with your code.

Line 10 — COPY . .

```
COPY . . # copy everything else into /app
```

Now the real code goes in: `app.py`, `templates/index.html`, `message.sql`.

The first `.` is the source — the **build context**, the folder you pointed `docker build` at. The second `.` is the destination, which thanks to `WORKDIR` means `/app`.

WATCH OUT This repository has **no** `.dockerignore` file, and that is a genuine gap. `COPY . .` therefore also copies the `.git` folder (your entire commit history), the `k8s/` and `eks-manifests/` folders, the `Jenkinsfile`, and — most importantly — the `mysql-data/` directory that `docker-compose` creates on your machine to hold the database files. That last one can be hundreds of megabytes of database state baked into your application image.

Creating a `.dockerignore` beside the Dockerfile fixes it. A reasonable starting point:

```
.git
.gitignore
mysql-data/
k8s/
eks-manifests/
__pycache__/
*.pyc
.env
```

KEY IDEA `.dockerignore` and the "dependencies before code" ordering are a **matched pair**. The ordering makes the cache useful; `.dockerignore` stops stray local files from destroying it.

Line 11 — CMD ["python", "app.py"]

```
CMD ["python", "app.py"] # what to run when a container starts
```

`CMD` sets the **default command** for containers made from this image. Nothing runs during the build — this is a note left for later.

`python app.py` starts Flask. The last line of `app.py` is what makes it work in a container:

```
app.run(host='0.0.0.0', port=5000, debug=True)
```

`host='0.0.0.0'` means **listen on every network interface**. This is essential. The default, `127.0.0.1`, accepts connections only from inside the container itself, so the port mapping would work but nothing would ever answer. Every manifest in Parts III and IV refers to port **5000** because of this line.

The square brackets are the **exec form**, and they are the form you want: `python` becomes the container's main process (PID 1) and receives shutdown signals directly, so the container stops cleanly. The shell form, `CMD python app.py`, wraps it in `/bin/sh -c`, which swallows those signals and makes containers take ten extra seconds to die.

WATCH OUT `debug=True` ships the Werkzeug interactive debugger. If an unhandled exception occurs, anyone who can reach the page gets a web console that **executes arbitrary Python on your server**. On a NodePort or a LoadBalancer that is a remote code execution hole. It is fine on your laptop and genuinely dangerous anywhere else — set it from an environment variable and default it to off.

The Dockerfile at a glance

Step	Instruction	Runs when?	Why it is here
1	<code>FROM python:3.9-slim</code>	build	Linux + Python 3.9, small base
2	<code>WORKDIR /app</code>	build	Sets the working folder for everything below
3	<code>RUN apt-get ...</code>	build	Compiler + MySQL headers, index cleaned up in the same layer
4	<code>COPY requirements.txt .</code>	build	Dependency list only — changes rarely
5	<code>RUN pip install ...</code>	build	The slow step, cached until requirements change
6	<code>COPY . .</code>	build	The app source — changes constantly
7	<code>CMD ["python", "app.py"]</code>	run	Starts Flask on port 5000

5. Dockerfile-multistage — the Slim Version

From the repo: [Dockerfile-multistage](#)

The repo ships a second, better Dockerfile. It builds the same app but produces a much smaller and safer image, and it demonstrates the single most useful Dockerfile technique there is.

```
# ----- Stage 1: Build Stage -----
FROM python:3.9 AS builder

WORKDIR /app

# Install build dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc default-libmysqlclient-dev pkg-config && \
    rm -rf /var/lib/apt/lists/*

# Copy and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ----- Stage 2: Final Stage -----
FROM python:3.9-slim

WORKDIR /app

# Install runtime dependencies only
RUN apt-get update && \
    apt-get install -y --no-install-recommends libmariadb3 && \
    rm -rf /var/lib/apt/lists/*

# Copy dependencies and application code from the builder stage
COPY --from=builder /usr/local/lib/python3.9/site-packages/
/usr/local/lib/python3.9/site-packages/
COPY . .
```

```
CMD ["python", "app.py"]
```

The problem it solves

The Dockerfile in chapter 4 installs `gcc`, the MySQL development headers and `pkg-config` — and then ships all of them to production. They are needed for about ninety seconds, to compile `mysqlclient`, and are then dead weight forever. Worse, a compiler inside a production container is a useful tool for anyone who breaks into it.

A **multi-stage build** fixes this. You use the heavy tools to *produce* your app, then copy only the finished result into a clean image and throw the tools away.

Stage 1 — the builder

```
FROM python:3.9 AS builder # the FULL image: compilers and headers included
```

Two things are new here.

`AS builder` gives this stage a **name**. Nothing else changes about it — the name simply lets a later stage refer back to it.

The base is `python:3.9`, not `-slim`. The full image is used on purpose: this stage's only job is compiling, so the extra 800 MB of tooling is welcome. None of it will survive.

```
RUN apt-get install -y --no-install-recommends gcc default-libmysqlclient-dev pkg-config
```

The same build tools as before. `--no-install-recommends` is a small improvement over chapter 4: `apt` normally also installs "recommended" extras that are nice-to-have rather than required, and this flag skips them.

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

Dependencies get installed into `/usr/local/lib/python3.9/site-packages/`. Note there is no `COPY . .` in this stage — the application code is never needed here, because only the libraries are being built.

Stage 2 — the final image

```
FROM python:3.9-slim # a SECOND FROM: everything above is now discarded
```

A second `FROM` starts a brand-new image from scratch. Everything in stage 1 — the compilers, the headers, the `apt` index — is gone unless explicitly copied forward.

```
RUN apt-get install -y --no-install-recommends libmariadb3
```

One runtime package. `mysqlclient` compiled against the MySQL client library, so the compiled module needs that library *present* to load — but it needs the library only, not the development headers. `libmariadb3` is the compatible runtime library, a few megabytes instead of a few hundred.

NOTE `default-libmysqlclient-dev` (build time) contains headers and a compiler target; `libmariadb3` (run time) contains just the shared library the finished module loads. Pairing a `-dev` package in the builder with its runtime-only equivalent in the final stage is the standard pattern for any compiled dependency.

```
COPY --from=builder /usr/local/lib/python3.9/site-packages/
/usr/local/lib/python3.9/site-packages/
```

This is the line that makes it all work. `COPY --from=builder` copies out of the earlier *stage* rather than from your machine, lifting the fully installed Python packages — including the compiled `mysqlclient` — into

the clean image. The compiler that produced them stays behind.

```
COPY . .  
CMD ["python", "app.py"]
```

Then the application code, and the same start command as before.

What it buys you

	Dockerfile	Dockerfile-multistage
Final base	python:3.9-slim	python:3.9-slim
Contains gcc and dev headers	Yes	No
Rough final size	~450 MB	~180 MB
Attack surface	Compiler available to an intruder	Runtime only

KEY IDEA `COPY --from=<stage>` lets you use heavy tools to *build* your app without shipping those tools *with* your app. It is the single highest-value line you can add to a Dockerfile, and it applies far beyond Python — Go, Java, Node and React images all use the same trick.

Two caveats in this particular file

WATCH OUT Copying [site-packages](#) alone misses `/usr/local/bin`. Any dependency that installs a command-line script — [flask](#), [gunicorn](#), [celery](#) — would be absent from the final image. It works here only because the app is started with `python app.py` rather than a console script. If you later switch to [gunicorn](#), you must copy `/usr/local/bin` as well.

NOTE Both stages must use the **same Python minor version**. The copy targets a hard-coded `python3.9` path, so changing the builder to `python:3.11` while leaving the final stage at `python:3.9-slim` would copy into a directory Python never reads, and every import would fail.

To build with this file instead of the default one, point `-f` at it:

```
docker build -f Dockerfile-multistage -t two-tier-flask-app:slim . # -f picks a non-default Dockerfile  
docker images two-tier-flask-app # compare the two sizes
```

6. docker-compose.yml — Both Tiers on One Machine

From the repo: [docker-compose.yml](#)

Before Kubernetes, there is a much simpler way to run two tiers together: **Docker Compose**. One file describes both containers, the network between them and the storage, and one command starts everything.

```
version: "3.8"  
  
services:  
  mysql:  
    user: "${UID}:${GID}"  
    image: mysql:5.7  
    container_name: mysql  
    environment:  
      MYSQL_ROOT_PASSWORD: root
```

```

    MYSQL_DATABASE: devops
    MYSQL_USER: admin
    MYSQL_PASSWORD: admin
volumes:
  - ./mysql-data:/var/lib/mysql
  - ./message.sql:/docker-entrypoint-initdb.d/message.sql
networks:
  - twotier
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 60s

flask-app:
  image: nitin1094/two-tier-flask-app:latest
  container_name: flask-app
  ports:
    - "5000:5000"
  environment:
    MYSQL_HOST: mysql
    MYSQL_USER: root
    MYSQL_PASSWORD: root
    MYSQL_DB: devops
  depends_on:
    - mysql
  networks:
    - twotier
  restart: always
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:5000/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s

networks:
  twotier:

```

The shape of the file

Three top-level keys, and that is the whole structure:

Key	What it holds
<code>version: "3.8"</code>	The Compose file format version
<code>services:</code>	One block per container — here <code>mysql</code> and <code>flask-app</code>
<code>networks:</code>	The private networks containers are attached to

NOTE `version:` is obsolete in modern Docker Compose (v2) and you may see a warning telling you so. It is harmless — Compose ignores it and works out the format itself. You can safely delete the line.

The mysql service

```
image: mysql:5.7 # pull the official MySQL 5.7 image from Docker Hub
container_name: mysql # fix the container's name instead of letting Compose generate one
```

`container_name: mysql` matters more than it looks: it is also the **hostname** other containers on the same network use to reach it. That is what makes `MYSQL_HOST: mysql` work in the other service.

```
environment:
  MYSQL_ROOT_PASSWORD: root # password for the "root" superuser
  MYSQL_DATABASE: devops # create this database on first start
  MYSQL_USER: admin # also create this ordinary user...
  MYSQL_PASSWORD: admin # ...with this password
```

These four variables are read by the official MySQL image's startup script the **first time** it runs, and they create the database and the accounts.

WATCH OUT Notice the mismatch. Compose creates a user `admin / admin`, but the Flask service below connects as `root / root`. Both work — `root` exists with the password `root` — but the `admin` account is created and then never used. This kind of leftover is a common source of confusion when you later try to tighten permissions.

```
volumes:
  - ./mysql-data:/var/lib/mysql
  - ./message.sql:/docker-entrypoint-initdb.d/message.sql
```

Two **bind mounts**, each in the form `host-path:container-path`:

The first keeps the database files in a `mysql-data/` folder next to the compose file. Without it, `docker compose down` would destroy every message. This is also the folder that `.dockerignore` should be excluding, as chapter 4 warned.

The second is a neat trick. The MySQL image runs any `.sql` file it finds in `/docker-entrypoint-initdb.d/` on first startup, so mounting `message.sql` there creates the table automatically:

```
CREATE TABLE messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  message TEXT
);
```

NOTE Both of these run **only when the data directory is empty**. If you change `MYSQL_ROOT_PASSWORD` later, or edit `message.sql`, nothing happens — the directory is no longer empty, so MySQL skips initialisation entirely. To genuinely start over you must delete `mysql-data/` first. This catches almost everyone at least once.

```
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
  interval: 10s # run the check every 10 seconds
  timeout: 5s # a check that takes longer than 5s counts as failed
  retries: 5 # 5 failures in a row marks the container "unhealthy"
  start_period: 60s # ignore failures during the first 60s while MySQL initialises
```

A **health check** is a command Docker runs inside the container to decide whether it is actually working, as opposed to merely running. `mysqladmin ping` asks the database "are you accepting connections?".

`start_period: 60s` is the important tuning knob. MySQL takes a while to initialise on first start, and without a grace period it would be declared unhealthy before it ever had a chance.

The flask-app service

```
image: nitin1094/two-tier-flask-app:latest # a pre-built image from Docker Hub
ports:
  - "5000:5000" # host port 5000 -> container port 5000
```

`ports` is what makes the app reachable from your browser at <http://localhost:5000>. The **left** number is the port on your machine, the **right** is the port inside the container. They happen to match here, but `"8080:5000"` would serve the same app on `localhost:8080`.

NOTE This service has no `build:` key, so Compose **pulls** the image rather than building it from the Dockerfile you just read. To run your own local build instead, either add `build: .` beneath `image:`, or build and tag it with that exact name first. This detail matters again in chapter 8, where the Jenkins pipeline trips over it.

```
environment:
  MYSQL_HOST: mysql # the OTHER service's container_name
  MYSQL_USER: root
  MYSQL_PASSWORD: root
  MYSQL_DB: devops
```

Here are the four variables from chapter 1, finally filled in. `MYSQL_HOST: mysql` is the key line: Docker's built-in DNS resolves the name `mysql` to that container's IP on the shared network. Exactly the same idea as Kubernetes service discovery, one step smaller.

```
depends_on:
  - mysql
```

Start `mysql` before `flask-app`.

WATCH OUT `depends_on` waits for the container to **start**, not for MySQL to be **ready to accept connections**. MySQL takes 30–60 seconds to initialise, so on a cold start Flask will try to connect, fail, and — thanks to `restart: always` — be restarted by Docker until MySQL is finally up. It self-heals, but the first minute of logs looks alarming. The fix is `depends_on: { mysql: { condition: service_healthy } }`, which uses the health check you already defined.

```
restart: always
```

Restart this container whenever it stops, including after a reboot. This is what quietly rescues the startup race above.

```
healthcheck:
  test: ["CMD-SHELL", "curl -f http://localhost:5000/health || exit 1"]
```

WATCH OUT This health check **cannot succeed**, for two independent reasons. First, `app.py` defines only two routes — `/` and `/submit` — so there is no `/health` endpoint and `curl -f` returns a failure on the 404. Second, `curl` is not installed in `python:3.9-slim`, so the command is not even found. The container therefore reports **unhealthy** forever. Nothing depends on that status here, so the app still works — but it is misleading, and it would break any tooling that gates on health. Either add a `/health` route to `app.py` and install `curl` in the Dockerfile, or delete the block.

The network

```
networks:
  twotier:
```

Declares a private network named `twotier`; both services join it via their `networks:` key. Containers on a shared user-defined network can reach each other **by name** and are isolated from containers outside it. Defining it with no options accepts all the defaults, which is exactly right here.

Running it

```
docker compose up -d # start both services in the background
docker compose ps # see status and health of each container
docker compose logs -f flask-app # follow the app's logs
docker compose down # stop and remove containers (mysql-data/ survives)
```

WATCH OUT The very first line of the `mysql` service is `user: "${UID}:${GID}"`, which tells Docker to run MySQL as your own user instead of root, so the files in `mysql-data/` belong to you. The catch: in `bash`, `UID` is a *shell* variable and `GID` usually does not exist at all — neither is exported to Compose, so both expand to empty and the value becomes a bare `":"`. Export them first, or the container may fail to start: `export UID=$(id -u) GID=$(id -g)` before `docker compose up`.

7. The Makefile — Shortcuts for Compose

From the repo: [Makefile](#)

A small convenience wrapper. `make` is a decades-old tool that runs named command recipes, and here it simply wraps the Compose commands so you can type `make run` instead of remembering flags.

```
# Determine the operating system
OS := $(shell uname)

# Define Docker compose command
DOCKER_COMPOSE := docker-compose

# Define Docker compose service name
SERVICE_NAME := web
```

`$(shell uname)` runs `uname` and stores the answer — `Linux`, `Darwin` (macOS) or `Windows_NT`. The other two lines are plain variables.

```
build:
ifeq ($(OS),Linux)
@echo "Building for Linux"
$(DOCKER_COMPOSE) build
endif
```

A **target** named `build`, run with `make build`. The `ifeq` blocks pick the right commands for the detected operating system; Linux and macOS both run `docker-compose build`, and Windows is left as a friendly stub.

The `@` prefix on `echo` means "run this without printing the command itself" — without it, `make` echoes every line before running it.

The remaining targets are equally short:

Target	What it runs	In plain terms
<code>make build</code>	<code>docker-compose build</code>	Build the images
<code>make run</code>	<code>docker-compose up -d</code>	Start everything in the background
<code>make stop</code>	<code>docker-compose down</code>	Stop and remove the containers
<code>make clean</code>	<code>down</code> , then <code>rm -f</code> , then <code>docker system prune -f</code>	Stop, remove, and delete unused Docker data

```
clean: stop
$(DOCKER_COMPOSE) rm -f
docker system prune -f
```

`clean: stop` means "`clean` depends on `stop`" — `make` runs the `stop` target first, automatically. That dependency arrow is really the only thing `make` does that a shell script does not.

WATCH OUT `docker system prune -f` deletes **all** unused containers, networks, images and build cache on your machine — not only this project's. `-f` skips the confirmation prompt. If you use Docker for anything else, `make clean` will take that with it.

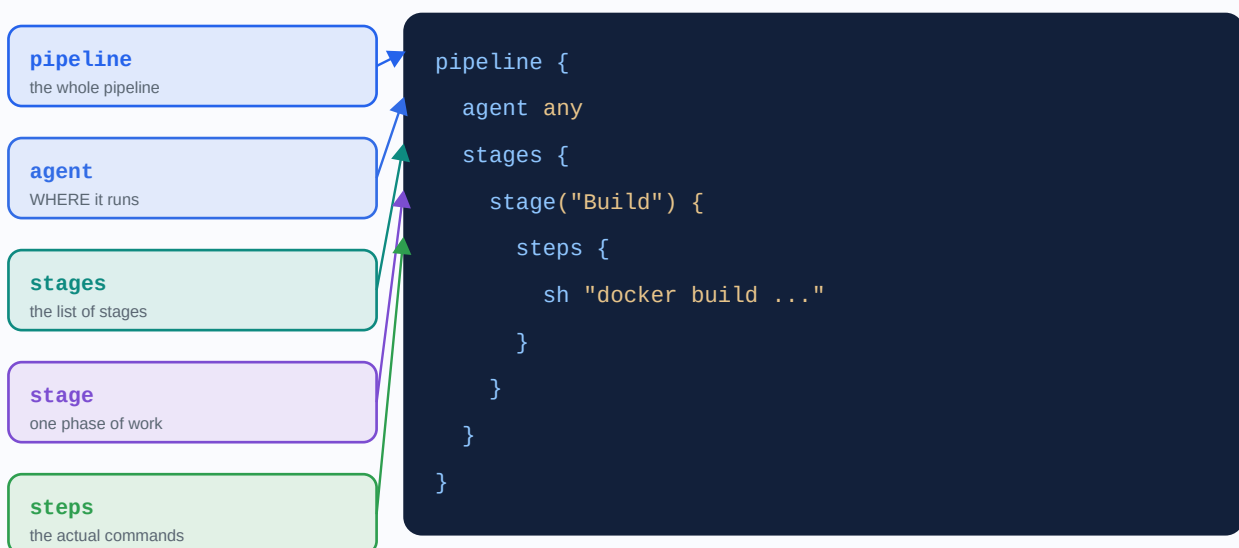
NOTE Two small staleness issues. `DOCKER_COMPOSE := docker-compose` uses the old hyphenated v1 command; modern Docker ships `docker compose` as a subcommand, so on a current install these targets fail with "command not found". And `SERVICE_NAME := web` names a service that does not exist in `docker-compose.yml` (which defines `mysql` and `flask-app`) — it is simply unused.

8. The Jenkinsfile — Automating Build and Deploy

From the repo: [Jenkinsfile](#)

The last root file. A **Jenkinsfile** describes a CI/CD pipeline: the steps Jenkins runs automatically every time you push code — clone, scan, build, push, deploy.

Anatomy of a Declarative Pipeline (the Jenkinsfile)



```

@Library("Shared") _
pipeline{

    agent { label "dev"};

    stages{
        stage("Code Clone"){
            steps{
                script{
                    clone("https://github.com/nitin1094/two-tier-flask-app-k8s.git", "main")
                }
            }
        }
        stage("Trivy File System Scan"){
            steps{
                script{
                    trivy_fs()
                }
            }
        }
        stage("Build"){
            steps{
                sh "docker build -t two-tier-flask-app ."
            }
        }
        stage("Test"){
            steps{
                echo "Run test suite here"
            }
        }
        stage("Push to Docker Hub"){
            steps{
                script{
                    docker_push("dockerHubCreds","two-tier-flask-app")
                }
            }
        }
        stage("Deploy"){
            steps{
                sh "docker compose up -d --build flask-app"
            }
        }
    }
}

```

The skeleton

Read the nesting from the outside in — it is always the same four levels:

```
pipeline { } the whole pipeline
agent { } WHERE it runs
stages { } the list of stages
    stage("X") { } one phase of work
        steps { } the actual commands
```

```
@Library("Shared") _
```

```
@Library("Shared") _
```

Loads a **Jenkins shared library** called `Shared` — a separate Git repository holding reusable functions. The lonely underscore is required Groovy syntax; it is the "thing" the annotation attaches to, and it means "apply this to the whole script".

This one line is why `clone(...)`, `trivy_fs()` and `docker_push(...)` can be called later as if they were built in. They are not — they are functions defined in that library.

WATCH OUT This pipeline **will not run** on a Jenkins instance that does not have a shared library named exactly `Shared`, configured under *Manage Jenkins to System to Global Pipeline Libraries*, containing those three functions. The failure is immediate and the error message ("No such DSL method") does not make the cause obvious. Replace the calls with plain `sh` steps if you do not have the library.

```
agent { label "dev" }
```

```
agent { label "dev"};
```

Where the pipeline runs. Jenkins can manage many build machines ("agents"), each tagged with labels. This says: run on any agent labelled `dev`. That machine needs Docker and Docker Compose installed, since every stage shells out to them.

`agent any` would mean "any available agent", and is what you would use on a single-machine Jenkins.

The six stages

Stage	What it does
Code Clone	Clones the repo at branch <code>main</code> using the shared library's <code>clone()</code>
Trivy File System Scan	Runs Trivy, an open-source scanner, over the checked-out files to report known vulnerabilities and leaked secrets
Build	<code>docker build -t two-tier-flask-app .</code> — the Dockerfile from chapter 4
Test	A placeholder that only echoes a message
Push to Docker Hub	Tags and pushes the image using the stored credential <code>dockerHubCreds</code>
Deploy	<code>docker compose up -d --build flask-app</code>

Two of those deserve a closer look.

```
stage("Test"){
    steps{
        echo "Run test suite here"
    }
}
```

An honest placeholder. It always passes, so it provides no safety at all — but it keeps the slot visible so the next person knows where tests belong. Replacing that `echo` with `sh "pytest"` is the single most valuable improvement you could make to this pipeline.

```
docker_push("dockerHubCreds", "two-tier-flask-app")
```

`dockerHubCreds` is not the password — it is the **ID of a credential stored in Jenkins**, under *Manage Jenkins to Credentials*. Jenkins injects the real username and token at run time and masks them in the logs. This is the right pattern: the secret never appears in the repository.

The Deploy stage does not do what it looks like

```
sh "docker compose up -d --build flask-app"
```

WATCH OUT `--build` tells Compose to rebuild the `flask-app` service from source — but as chapter 6 noted, that service has **no `build: key`**, only `image:.` So there is nothing to build, and Compose starts the container from `nitin1094/two-tier-flask-app:latest` pulled from Docker Hub. Meanwhile the Build stage produced a local image tagged `two-tier-flask-app` — a *different* name. The result: the pipeline deploys whatever was last pushed to Docker Hub, which is usually what you wanted, but not for the reason the code suggests. Adding `build: .` to the `flask-app` service makes the intent and the behaviour agree.

NOTE Deploying with `docker compose` also means this pipeline deploys to the **Jenkins agent itself**, not to a Kubernetes cluster. To deploy the manifests in Parts III and IV you would swap this stage for `sh "kubectl apply -f k8s/"` and give the agent a kubeconfig.

The post block

```
post{
    success{
        script{
            emailx from: '<your-email>',
            to: '<your-email>',
            body: 'Build success for Demo CICD App',
            subject: 'Build success for Demo CICD App'
        }
    }
    failure{
        script{
            emailx from: '<your-email>',
            to: '<your-email>',
            body: 'Build Failed for Demo CICD App',
            subject: 'Build Failed for Demo CICD App'
        }
    }
}
```

`post` runs **after** all stages finish, and its sub-blocks are conditional: `success` only when everything passed, `failure` only when something broke. `emailx` is the Email Extension plugin.

NOTE `<your-email>` is a placeholder you must replace in both blocks. The pipeline also needs an SMTP server configured in Jenkins, or these steps fail *after* an otherwise successful build — confusingly marking the whole run as failed.

TIP `post` also supports `always` (run no matter what, ideal for cleanup) and `unstable` (tests failed but the build did not). A common pattern is `always { sh "docker system prune -f" }` to stop build agents filling up their disks.

PART III

The k8s/ Folder (a self-managed cluster)

Seven manifests for a kubeadm-style cluster: a `hostPath` PV and Claim, two Deployments, a bare Pod for contrast, and a `NodePort` Service.

9. How to Read the k8s/ Manifests

From the repo: [k8s/](#)

Seven YAML files that deploy the app on a cluster you run yourself — one built with `kubeadm`, or Minikube, or kind. Before opening them one at a time, here is the map and the rules that apply to all of them.

The order they must be applied in

Kubernetes objects depend on each other. A pod that mounts a volume cannot start until the volume exists. So you work from the bottom of the stack upwards — storage, then database, then app:

```
# 1) Persistent storage for MySQL
kubectl apply -f k8s/mysql-pv.yml
kubectl apply -f k8s/mysql-pvc.yml

# 2) The database tier
kubectl apply -f k8s/mysql-deployment.yml
kubectl apply -f k8s/mysql-svc.yml

# 3) The application tier
kubectl apply -f k8s/two-tier-app-deployment.yml
kubectl apply -f k8s/two-tier-app-svc.yml
```

TIP Order matters less than it appears, because Kubernetes keeps retrying. Apply the app first and its pod simply crash-loops for a minute until MySQL exists, then settles. Getting the order right just saves you a wall of alarming-looking errors.

NOTE `kubectl apply -f k8s/` applies every `.yaml` and `.yml` file in the folder in one go. That works here because all seven files are flat in one directory — unlike a repo with per-tier sub-folders, where `apply -f` would silently skip them (it does not recurse without `-R`).

Rule one — everything lands in the `default` namespace

None of these manifests sets a `namespace:`, so every object goes into `default`. That is why no `kubectl` command in this Part carries a `-n` flag.

```
kubectl get pods # no -n needed: these objects are in "default"
kubectl get svc
```

Rule two — two labels do all the wiring

```
app: mysql # worn by the MySQL pod; searched for by the mysql Deployment and Service
app: two-tier-app # worn by the Flask pods; searched for by their Deployment and Service
```

Nothing in this folder is connected by IP address. Every link — Deployment to pod, Service to pod — is a label match.

The seven files at a glance

File	Kind	Purpose
mysql-pv.yml	PersistentVolume	Offers 256 MiB of storage from a folder on the node
mysql-pvc.yml	PersistentVolumeClaim	Requests 256 MiB on the pod's behalf
mysql-deployment.yml	Deployment	Runs one MySQL pod and mounts the claim
mysql-svc.yml	Service	A stable internal name, <code>mysql</code> , for the database
two-tier-app-pod.yml	Pod	A bare pod — an alternative to the Deployment, see chapter 13
two-tier-app-deployment.yml	Deployment	Runs the Flask pod properly
two-tier-app-svc.yml	Service	Exposes the app outside the cluster on a NodePort

NOTE `two-tier-app-pod.yml` and `two-tier-app-deployment.yml` both create the same Flask container. They are **alternatives**, not a pair. Apply only one — chapter 13 explains which and why.

10. Storage — `mysql-pv.yml` and `mysql-pvc.yml`

From the repo: [k8s/mysql-pv.yml](#)

These two short files solve one problem: **containers forget everything when they restart**.

A container's filesystem is temporary. Delete the pod and its files go with it — fine for a stateless web app, unacceptable for a database. A **volume** is storage that outlives the pod.



Pods are disposable, so their data must live outside them. The pod claims a PVC, the PVC binds to a PV (real storage) — now MySQL's data survives a pod restart.

The supply side — `mysql-pv.yml`

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 256Mi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /home/ubuntu/two-tier-flask-app/mysqldata #This is your host path where your data
will be stored. Make sure to create mysqldata directory in mentioned path

```

A **PersistentVolume (PV)** represents an actual piece of storage in the cluster. Think of it as the disk itself.

Field	What it means
<code>capacity.storage: 256Mi</code>	Offers 256 mebibytes. Note Mi (1024-based), not M (1000-based).
<code>volumeMode: Filesystem</code>	Present it as a mountable directory rather than a raw block device — the usual choice.
<code>accessModes: [ReadWriteOnce]</code>	Mountable read-write by one node at a time.
<code>persistentVolumeReclaimPolicy: Retain</code>	When the claim is deleted, keep the data .
<code>hostPath.path</code>	Where the bytes physically live: that folder on whichever node the pod runs on.

Retain is worth pausing on. The alternative, **Delete**, wipes the underlying storage when the PVC is removed. **Retain** keeps it and marks the PV as **Released**, which protects you from losing a database to a careless `kubectl delete pvc` — at the cost of needing a manual clean-up before that PV can be reused.

WATCH OUT `hostPath` means "a folder on the node's own disk", and it is the least production-suitable volume type there is. If the pod is rescheduled onto a **different** node it finds an empty directory there and, as far as the application is concerned, all the data has vanished. If the node is terminated, the data is gone for good. It is fine for a single-node learning cluster and nothing else.

WATCH OUT The comment in the file is an instruction, not a description. You must create `/home/ubuntu/two-tier-flask-app/mysqldata` **on the worker node** before applying this, or the MySQL pod fails to start. On a multi-node cluster you would have to create it on every node the pod might land on — which is exactly why `hostPath` does not scale.

The demand side — `mysql-pvc.yml`

From the repo: [k8s/mysql-pvc.yml](#)

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 256Mi

```

A **PersistentVolumeClaim (PVC)** is a *request* for storage. The pod asks for what it needs; Kubernetes finds a PV that satisfies the request and **binds** the two together.

Field	What it means
<code>name: mysql-pvc</code>	The name the MySQL Deployment references in its <code>volumes:</code> block
<code>accessModes: [ReadWriteOnce]</code>	Must be compatible with the PV's mode, or the two will never bind
<code>resources.requests.storage: 256Mi</code>	Asks for 256 MiB. The PV offers exactly that, so it qualifies.

KEY IDEA PV and PVC are a **supply and demand** pair, like a job advert and an application. The PV says "256 MiB available here"; the PVC says "256 MiB wanted". Kubernetes plays matchmaker. The pod only ever names the **claim** — it never needs to know where the storage actually is.

The subtle trap: no `storageClassName`

This PVC does not set `storageClassName` at all, and on a modern cluster that changes everything.

Cluster	What happens
Plain <code>kubeadm</code> , no default <code>StorageClass</code>	No dynamic provisioning is possible, so the PVC binds to <code>mysql-pv</code> — the intended behaviour
Minikube, kind, any managed cluster	A default <code>StorageClass</code> exists, so Kubernetes provisions a brand-new volume and ignores <code>mysql-pv</code> entirely

In the second case everything still appears to work — the PVC reaches `Bound`, the pod starts, the app runs — but your data is somewhere you did not choose, and `mysql-pv` sits unused at `Available` forever.

TIP To force the binding to `mysql-pv` on any cluster, add one line to the PVC: `storageClassName: ""`. The empty string explicitly means "do not dynamically provision — bind me to an existing PV". Leaving the field out is *not* the same thing. You can tell which happened with `kubectl get pvc mysql-pvc` and check the `VOLUME` column: it should read `mysql-pv`, not `pvc-<random-uuid>`.

11. mysql-deployment.yml

From the repo: [k8s/mysql-deployment.yml](https://github.com/kubernetes/examples/blob/master/mysql-deployment.yml)

The database itself. This is the first full Deployment, so we will take it slowly — the structure repeats for the app tier.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql

```

```

labels:
  app: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:latest
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "admin"
            - name: MYSQL_DATABASE
              value: "mydb"
            - name: MYSQL_USER
              value: "admin"
            - name: MYSQL_PASSWORD
              value: "admin"
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: mysqldata
              mountPath: /var/lib/mysql # this is your container path from where your data
will be stored
          volumes:
            - name: mysqldata
              persistentVolumeClaim:
                claimName: mysql-pvc # PVC claim name

```

The two spec blocks — the thing that confuses everyone

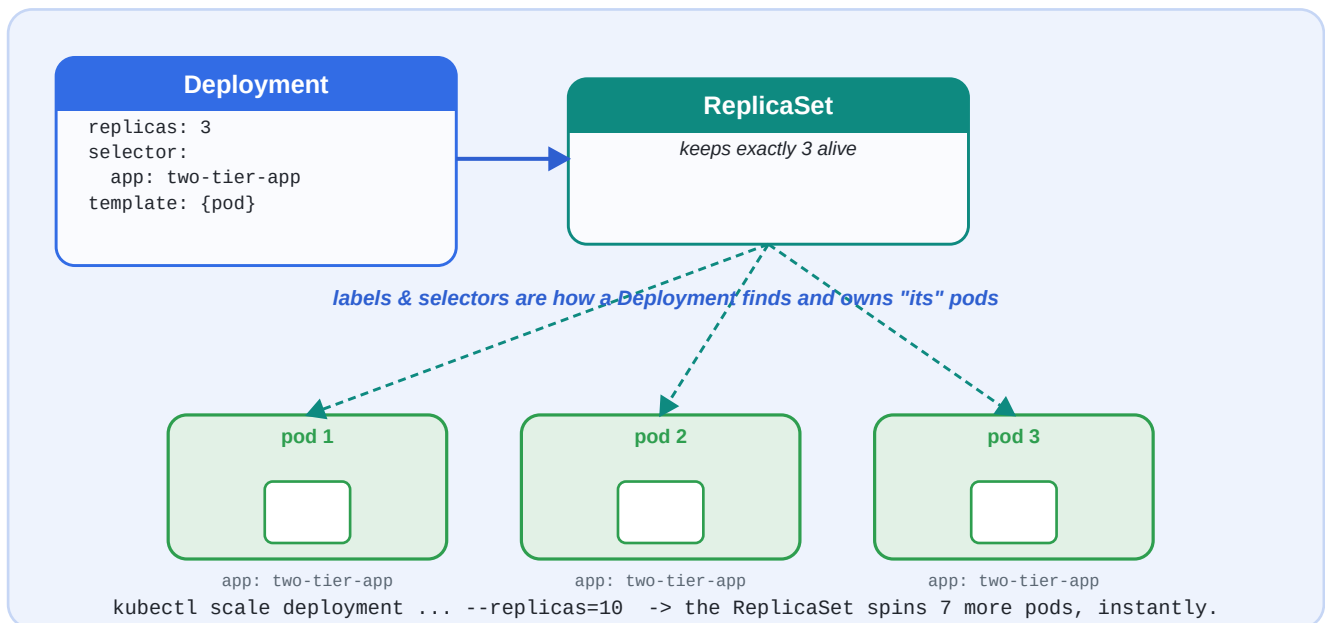
Look carefully and you will see `spec:` **appears twice**, and so does `metadata:`. That is not a typo, and understanding it makes every Deployment readable:

```

spec: # <- the DEPLOYMENT's spec: how many pods, which pods are mine
  replicas: 1
  selector: ...
  template: # <- a pod BLUEPRINT starts here
    metadata: # <- the POD's metadata (its labels)
      labels: ...
    spec: # <- the POD's spec: containers, volumes
      containers: ...

```

The outer `spec` is about **managing pods**. Everything under `template:` is a complete pod definition — a stamp the Deployment uses to create each pod. Edit anything under `template` and the Deployment rolls out replacement pods.



replicas: 1 — and why it must stay 1

```
replicas: 1 # exactly one MySQL pod
```

For the Flask tier you could happily run several copies. For this database you must not, for two reasons:

- 1 The PV is `ReadWriteOnce` — only one node can mount it at a time.
- 2 Two `mysqld` processes writing the same data directory will corrupt it.

Scaling a database is not a matter of raising `replicas`; it needs replication, which in Kubernetes means a `StatefulSet` with per-pod storage rather than a `Deployment`.

NOTE A `Deployment` is designed for *stateless* workloads — interchangeable pods with no identity. Databases want stable names, stable storage and ordered startup, which is what a `StatefulSet` provides. Running MySQL as a one-replica `Deployment` is fine for a demo, and naming that trade-off is a good answer if an interviewer asks how you would harden this project.

selector and template.metadata.labels must match

```
selector:
  matchLabels:
    app: mysql # "the pods I manage wear this sticker"
template:
  metadata:
    labels:
      app: mysql # "...and here I am putting that sticker on them"
```

The `Deployment` finds its own pods by this label, and the `mysql` Service in chapter 12 searches for the very same one.

WATCH OUT These two must match, and `selector` is **immutable** after creation. If you later want to rename the label you cannot edit it in place — you must delete the `Deployment` and recreate it. Kubernetes rejects the update with a validation error.

The container

```
- name: mysql
  image: mysql:latest
```

`name: mysql` is just an identifier inside the pod; you use it with `kubectl logs <pod> -c mysql` when a pod has several containers.

WATCH OUT `mysql:latest` is an unpinned, **moving** tag. Today it is MySQL 9.x; when this repo was written it was 8.x. Two pods created a month apart can run different major versions of the database, and a MySQL major upgrade can refuse to open an older data directory. Compare this with `docker-compose.yml`, which sensibly pins `mysql:5.7`. Pin this too — `mysql:8.0` — and the surprise goes away.

The four environment variables

```
env:
  - name: MYSQL_ROOT_PASSWORD
    value: "admin" # password for the root superuser
  - name: MYSQL_DATABASE
    value: "mydb" # create this database on first start
  - name: MYSQL_USER
    value: "admin" # also create this ordinary user...
  - name: MYSQL_PASSWORD
    value: "admin" # ...with this password
```

The official MySQL image reads these on first startup and creates the database and accounts. Compare them with what the Flask app is told in chapter 14:

Variable	MySQL is told	Flask is told	Match?
Database	<code>MYSQL_DATABASE: mydb</code>	<code>MYSQL_DB: mydb</code>	yes
Username	<code>MYSQL_USER: admin</code>	<code>MYSQL_USER: root</code>	Flask uses root
Password	<code>MYSQL_ROOT_PASSWORD: admin</code>	<code>MYSQL_PASSWORD: admin</code>	yes, as root

So Flask connects as `root` with the password `admin`, which works. The `admin` user is created and then never used — the same leftover as in the compose file.

WATCH OUT These are plain text in a file you commit to GitHub. That is the single biggest difference between this folder and `eks-manifests/`, which moves the root password into a `Secret` (chapter 17). Even a `Secret` is only base64-encoded, but it is a step in the right direction and the correct habit to build.

NOTE All four variables take effect **only when the data directory is empty**. Because `/var/lib/mysql` is a persistent volume here, changing the password later does nothing — the directory is no longer empty, so MySQL skips initialisation entirely.

ports — documentation, mostly

```
ports:
  - containerPort: 3306 # MySQL's standard port
```

This declares that the container listens on 3306. Worth knowing: `containerPort` does **not** open or restrict anything — the process is reachable on that port whether or not you declare it. The field exists so humans

and tools reading the manifest know what the container does.

volumeMounts and volumes — the storage handshake

These two blocks always come as a pair, and they live at different levels:

```
volumeMounts: # inside the CONTAINER: where to attach it
  - name: mysqldata
    mountPath: /var/lib/mysql # MySQL's data directory
volumes: # inside the POD: what the volume actually is
  - name: mysqldata # the same name links the two
    persistentVolumeClaim:
      claimName: mysql-pvc # the PVC from chapter 10
```

Read it as two sentences:

- 1 **volumes** (pod level): "This pod has a volume called `mysqldata`; its contents come from the claim `mysql-pvc`."
- 2 **volumeMounts** (container level): "Attach the volume called `mysqldata` at `/var/lib/mysql` inside this container."

The shared string `mysqldata` joins them, and it is purely local to this pod — it has nothing to do with the PV's or PVC's names. So the full chain is:

```
mysqld writes to /var/lib/mysql -> mysqldata -> mysql-pvc (PVC) -> mysql-pv (PV) ->
/home/ubuntu/.../mysqldata on the node
```

KEY IDEA Storage in Kubernetes is deliberately indirect, and each hop earns its place. The container knows only a mount path. The pod knows only a claim name. The claim knows only a size and an access mode. Only the PV knows where the bytes really are — which is what lets you swap `hostPath` for a cloud disk without touching the Deployment at all.

What is missing: the table

Unlike `docker-compose.yml`, nothing here mounts `message.sql`, so the `messages` table is not created by MySQL. It is created by the application instead — `app.py` runs this on startup:

```
CREATE TABLE IF NOT EXISTS messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  message TEXT
);
```

That is why the app works on this cluster with no seed data at all. Chapter 18 shows the third approach, where a `ConfigMap` does the job.

12. mysql-svc.yml

From the repo: [k8s/mysql-svc.yml](#)

```

apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306

```

Six lines of `spec`, and they are what makes the two tiers able to find each other.

Why a Service is needed at all

Pods are disposable, and every time one restarts it gets a **new IP address**. If the Flask app hard-coded the MySQL pod's IP, it would break the first time that pod was rescheduled.

A **Service** solves this with a fixed name and a fixed virtual IP. It keeps a live list of the pods matching its selector and forwards traffic to them. The name never changes, however many times the pods behind it do.

Three types of Service

ClusterIP	NodePort	LoadBalancer
default	30000-32767	cloud only
Reachable ONLY inside the cluster.	Opens a port on every node's IP.	Cloud provisions a real load balancer + URL.
Use it for the database tier (pod-to-pod).	Reach the app at NodeIP:30007.	Best on EKS / managed Kubernetes.

Field by field

Field	What it means
<code>name: mysql</code>	Becomes a DNS name inside the cluster. This exact string is what <code>MYSQL_HOST: mysql</code> refers to.
<code>selector.app: mysql</code>	Forwards to every pod labelled <code>app: mysql</code> — the pod from chapter 11
<code>port: 3306</code>	The port the Service listens on
<code>targetPort: 3306</code>	The port on the pod to forward to

KEY IDEA The name of this Service is not cosmetic — it is the connection string. Rename it to `mysql-svc` and the app breaks instantly, because `MYSQL_HOST` in the app Deployment still says `mysql`. The two files are coupled by that one word.

No type: means ClusterIP

This Service does not specify a `type`, so it gets the default: `ClusterIP`. That means it is reachable **only from inside the cluster** — exactly right for a database. Nothing on the internet can reach MySQL, by design.

The DNS names it creates:

Name	Works from
<code>mysql</code>	any pod in the same namespace (<code>default</code>)
<code>mysql.default</code>	any pod in any namespace
<code>mysql.default.svc.cluster.local</code>	anywhere — the full canonical name

The app uses the short form, which is why this Service and the Flask pods must be in the same namespace.

TIP To prove a Service is wired up, run `kubectl get endpoints mysql`. If you see a pod IP and port, the selector matched. If you see `<none>`, it matched nothing — fix your labels before debugging anything else.

13. two-tier-app-pod.yml — and Why You Would Not Use It

From the repo: [k8s/two-tier-app-pod.yml](https://github.com/k8s/two-tier-app-pod.yml)

This file is here for teaching, and it is genuinely useful to read before the Deployment — because it shows you what a Deployment is actually managing.

```
apiVersion: v1
kind: Pod
metadata:
  name: two-tier-app-pod
spec:
  containers:
    - name: two-tier-app-pod
      image: nitin1094/flaskapp:latest
      env:
        - name: MYSQL_HOST
          value: "mysql" # MySQL Service name (resolved via cluster DNS)
        - name: MYSQL_PASSWORD
          value: "admin"
        - name: MYSQL_USER
          value: "root"
        - name: MYSQL_DB
          value: "mydb"
      ports:
        - containerPort: 5000
      imagePullPolicy: Always
```

A Pod is the raw unit

Notice how much **shorter** this is than a Deployment. There is no `replicas`, no `selector`, no `template`, and only one `spec`. You are describing a single pod directly, with nothing managing it.

Compare the shapes:

```
kind: Pod kind: Deployment
spec: spec:
  containers: ... replicas: 1
                selector: ...
                template:
                  spec:
                    containers: ... # <- the SAME block, nested two levels
deeper
```

The `containers:` block is identical in both. A Deployment is simply a wrapper that holds a pod blueprint plus instructions on how many to keep and how to replace them.

Why the Deployment wins

	Bare Pod	Deployment
Pod crashes	Container restarts, but if the pod is deleted it is gone for good	A replacement pod is created automatically
Node dies	The pod dies with it and never comes back	The pod is rescheduled onto a healthy node
Scaling	Copy the file and rename it, by hand	<code>kubectl scale --replicas=3</code>
Updating the image	Delete and recreate — downtime	A rolling update, no downtime
Rollback	None	<code>kubectl rollout undo</code>

KEY IDEA A pod has **no self-healing above the container level**. `kubectl delete pod two-tier-app-pod` deletes it permanently. Delete a pod owned by a Deployment and a new one appears within seconds — that is the whole point of a controller, and it is why you almost never write a bare Pod outside of a quick debugging session.

WATCH OUT Do not apply both this file and `two-tier-app-deployment.yml`. Both create pods labelled — well, almost. This bare pod has **no labels at all**, so `two-tier-app-service` (which selects `app: two-tier-app`) will **not** route to it. Applying only this file gives you a running pod that the Service cannot reach, and a page that never loads. If you want to experiment with the bare pod, add `labels: {app: two-tier-app}` under its `metadata`.

`imagePullPolicy: Always`

```
imagePullPolicy: Always # re-check the registry on every pod start
```

Tells the node to check Docker Hub for a newer image each time a pod starts, instead of reusing its cached copy. That is necessary here precisely **because** the tag is `:latest`: the tag stays the same while the image behind it changes, so a cache would serve stale code forever.

TIP With immutable tags — `flaskapp:v1.4.2`, or a Git commit SHA — you can use `imagePullPolicy: IfNotPresent` and get faster pod starts, because a given tag always means the same image. `Always` is a workaround for mutable tags, not a best practice in itself.

14. two-tier-app-deployment.yml

From the repo: [k8s/two-tier-app-deployment.yml](#)

The application tier, done properly.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: two-tier-app
  labels:
    app: two-tier-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: two-tier-app
  template:
    metadata:
      labels:
        app: two-tier-app
    spec:
      containers:
        - name: two-tier-app
          image: nitin1094/flaskapp:latest
          env:
            - name: MYSQL_HOST
              value: "mysql" # MySQL Service name (resolved via cluster DNS)
            - name: MYSQL_PASSWORD
              value: "admin"
            - name: MYSQL_USER
              value: "root"
            - name: MYSQL_DB
              value: "mydb"
          ports:
            - containerPort: 5000
          imagePullPolicy: Always
```

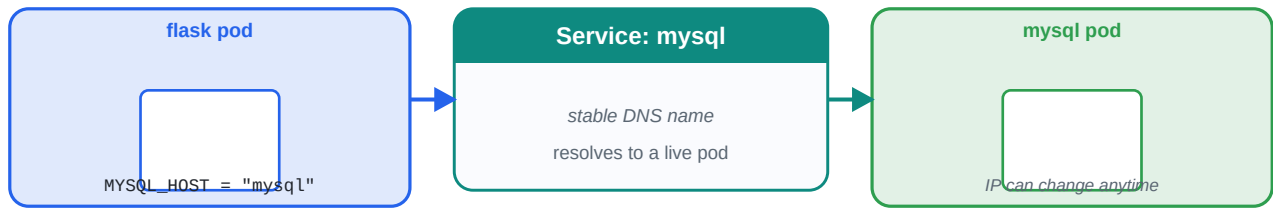
Structurally this is the Deployment you already read in chapter 11, so most of it needs no repeating. Three things are worth drawing out.

The four environment variables, at last

```
- name: MYSQL_HOST
  value: "mysql" # <- the Service name from chapter 12
- name: MYSQL_USER
  value: "root"
- name: MYSQL_PASSWORD
  value: "admin"
- name: MYSQL_DB
  value: "mydb"
```

These feed straight into the four `app.config['MYSQL_*']` lines from chapter 1. This is where the two tiers are joined, and `MYSQL_HOST: mysql` is the line that does it.

There is no IP address anywhere. The app asks cluster DNS for `mysql`, gets the Service's virtual IP, and the Service forwards to whichever pod currently wears `app: mysql`. Restart the database a hundred times and nothing here needs to change.



Never hard-code a pod's IP. Point to the service NAME and Kubernetes routes to a healthy pod.

KEY IDEA In Docker Compose the same trick worked through `container_name`. In Kubernetes it works through the **Service name**. Different mechanism, identical idea: never address another tier by IP — address it by a name that outlives the thing behind it.

replicas: 1 — but here you could raise it

Unlike MySQL, the Flask app is **stateless**: it keeps nothing in memory between requests and every pod talks to the same database. So any pod can serve any request, and `kubectl scale deployment/two-tier-app --replicas=3` would genuinely work — the Service would start load-balancing across all three.

That is the practical difference between the two tiers, and a good thing to be able to explain: stateless tiers scale horizontally by changing one number; stateful ones need far more care.

No health probes

This Deployment defines no `livenessProbe` and no `readinessProbe`. Both are simplifications.

Without a readiness probe, the pod is considered ready the moment its container starts — before Flask has finished booting and before MySQL is necessarily reachable. In practice that means a few seconds of failed requests after every rollout, and during a rolling update traffic can be sent to a pod that is not yet serving.

Adding one would be the single best improvement to this file:

```
readinessProbe: # "can this pod serve requests right now?"
  httpGet:
    path: /
    port: 5000
  initialDelaySeconds: 5
  periodSeconds: 5
```

NOTE A **readiness** probe failing removes the pod from its Service's endpoints — no restart, traffic simply goes elsewhere until it recovers. A **liveness** probe failing **restarts the container**. Always define readiness; add liveness only when you have a genuine hang scenario, and give it a generous `initialDelaySeconds` or a slow-starting pod will be killed in a loop.

The image is a different name

```
image: nitin1094/flaskapp:latest
```

WATCH OUT Note the name. The Jenkins pipeline in chapter 8 builds `two-tier-flask-app`, and `docker-compose.yml` runs `nitin1094/two-tier-flask-app:latest` — but these manifests use `nitin1094/flaskapp:latest`, a **third** name. They are separate Docker Hub repositories, so a build pushed by the pipeline does not update what Kubernetes pulls. Before deploying, either push your image under this name or edit `image:` in this file and in `two-tier-app-pod.yml` to match whatever you actually pushed.

15. two-tier-app-svc.yml — the NodePort

From the repo: [k8s/two-tier-app-svc.yml](https://github.com/stacks/ks/two-tier-app-svc.yml)

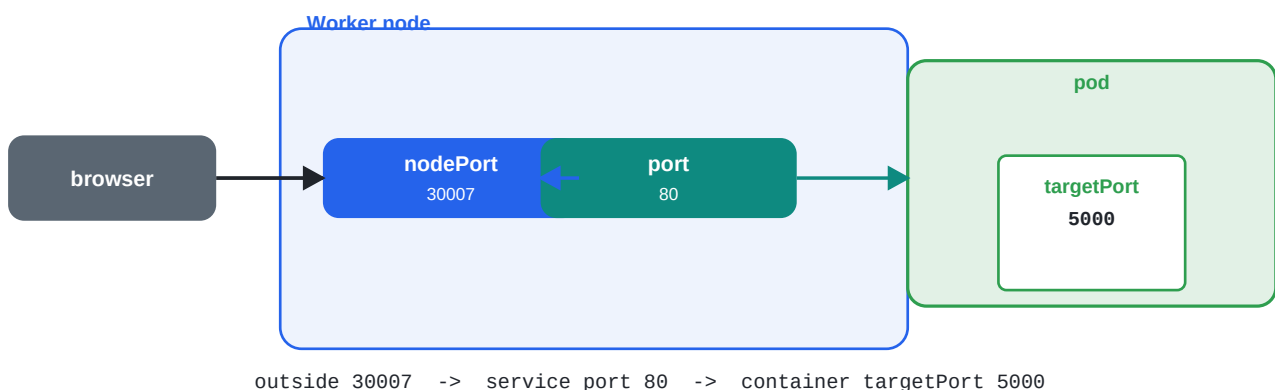
The last file in this folder, and the one that finally lets a browser in.

```
apiVersion: v1
kind: Service
metadata:
  name: two-tier-app-service
spec:
  selector:
    app: two-tier-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 30004
  type: NodePort
```

Three ports, and they all mean different things

This is the first Service in the guide with **three** port numbers, and confusing them is a rite of passage. Read them from the outside in:

How a NodePort Service maps its three ports



Field	Value here	Meaning
nodePort	30004	The port opened on every node's own IP , reachable from outside the cluster
port	80	The port the Service itself listens on, inside the cluster
targetPort	5000	The port on the pod — where Flask is actually listening

So the journey is:

```
browser -> <any-node-ip>:30004 -> Service :80 -> pod :5000 -> Flask
```

Open the app at <http://<worker-node-ip>:30004/>.

NOTE The figure above shows the generic pattern with a different node port; this project's actual numbers are the ones in the table — 30004, 80, 5000.

KEY IDEA `targetPort: 5000` must match `containerPort: 5000` in the Deployment, which must match `app.run(port=5000)` in `app.py`. Three files, one number. If the page never loads, walking that chain is the fastest way to find the break.

`type: NodePort`

```
type: NodePort
```

The three Service types, and when you would pick each:

Type	Reachable from	Use it for
ClusterIP	Inside the cluster only (the default)	Internal tiers — like <code>mysql</code> in chapter 12
NodePort	Outside, at <code><node-ip>:<30000-32767></code>	Demos and self-managed clusters with no load balancer
LoadBalancer	Outside, at a real cloud load balancer address	Managed clusters — this is what <code>eks-manifests/</code> uses

`NodePort` is the right choice here because a self-managed `kubeadm` cluster has no cloud load balancer to ask for. Its drawbacks are the ugly `IP:30004` URL, the restricted port range, and the fact that you must know a node's IP address.

WATCH OUT `nodePort` must fall within `30000-32767` by default; `30004` is valid. Pinning it is convenient — the URL never changes — but it will collide if another Service in the cluster claims the same number. Omit the field and Kubernetes assigns a free one, which you then read from `kubectl get svc`.

TIP Opening the port on the node is not always enough. On a cloud VM you must also allow inbound TCP 30004 in the **security group or firewall**, or the connection simply times out with no Kubernetes error at all. This is the most common reason a correctly-deployed NodePort app appears unreachable.

Verifying the whole folder

```
kubectl get pods # both pods should be Running and READY 1/1
kubectl get svc # mysql (ClusterIP) and two-tier-app-service (NodePort)
kubectl get pvc # mysql-pvc should say "Bound"
kubectl get endpoints # each service should list at least one pod IP
kubectl logs deployment/two-tier-app # Flask's startup output
kubectl exec -it deployment/mysql -- mysql -uroot -padmin -e "SHOW DATABASES;" # is mydb there?
```

PART IV

The eks-manifests/ Folder (Amazon EKS)

The same app on a managed cluster: a Secret, a ConfigMap that seeds the database, and a LoadBalancer Service in place of the NodePort.

16. What Changes on Amazon EKS

From the repo: [eks-manifests/](#)

Six YAML files that deploy the **same app** on **Amazon EKS** — Elastic Kubernetes Service, where AWS runs the control plane for you and your pods run on EC2 worker nodes.

The Kubernetes is identical. What changes is what the environment gives you, and this folder shows three of those differences.

The three differences, up front

	k8s/ (self-managed)	eks-manifests/ (EKS)	Why
Getting traffic in	<code>NodePort</code> on <code><node-ip>:30004</code>	<code>LoadBalancer</code> — a real AWS ELB with its own DNS name	EKS can provision cloud load balancers; a bare kubeadm cluster cannot
The root password	Plain text in the Deployment	Pulled from a <code>Secret</code>	A step towards keeping credentials out of the manifest
Creating the table	Left to <code>app.py</code> at startup	Seeded by a <code>ConfigMap</code> of SQL	Shows the ConfigMap pattern

And one difference that is **not** deliberate:

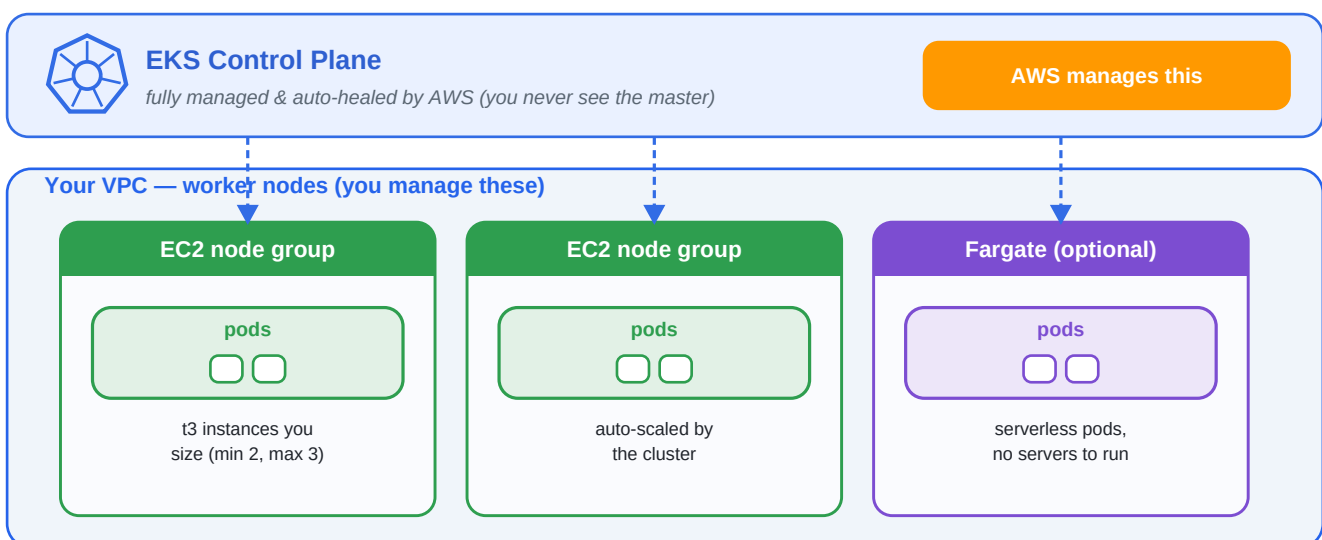
	k8s/	eks-manifests/
Database storage	<code>hostPath</code> PV + PVC	None at all

There is no PV and no PVC in this folder, and the MySQL Deployment mounts no data volume. Chapter 19 comes back to what that means.

The order to apply them in

```
kubectl apply -f eks-manifests/mysql-secrets.yml # 1. credentials first
kubectl apply -f eks-manifests/mysql-configmap.yml # 2. the seed SQL
kubectl apply -f eks-manifests/mysql-deployment.yml # 3. the database, which reads both
kubectl apply -f eks-manifests/mysql-svc.yml # 4. its stable name
kubectl apply -f eks-manifests/two-tier-app-deployment.yml
kubectl apply -f eks-manifests/two-tier-app-svc.yml # 5. the public load balancer
```

The Secret and the ConfigMap must exist **before** the Deployment that references them, or its pod sits in `CreateContainerConfigError` and never starts.



WATCH OUT Do not apply this folder on top of `k8s/`. The object names are identical — `mysql`, `two-tier-app`, `two-tier-app-service` — so the second apply overwrites the first and you get a confusing hybrid: a MySQL Deployment with no volume, still bound to a PVC that no longer has a mount. Pick one folder per cluster.

17. mysql-secrets.yml

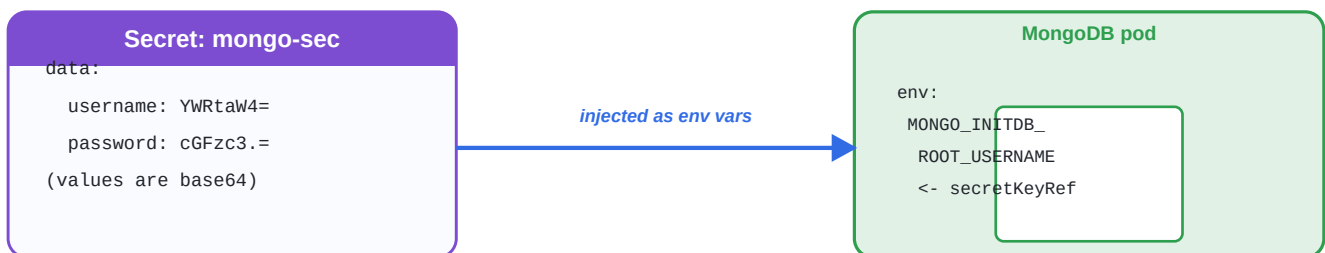
From the repo: [eks-manifests/mysql-secrets.yml](#)

The whole file is seven lines:

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque
data:
  MYSQL_ROOT_PASSWORD: YWRtaW4=
```

What a Secret is for

A **Secret** holds sensitive values — passwords, tokens, keys — so they do not have to be written inside your Deployment files. The Deployment just *refers* to this object by name, and Kubernetes injects the value into the container at startup.



A Secret keeps passwords out of your manifests; Kubernetes feeds them into the pod as environment variables.

Field	Meaning
<code>name:</code> <code>mysql-secret</code>	The handle the Deployment uses. <code>mysql-deployment.yml</code> references exactly this string.
<code>type:</code> <code>Opaque</code>	"Arbitrary user-defined key/value data" — the general-purpose Secret type
<code>data:</code>	The key/value pairs, with values base64-encoded

Base64 is encoding, not encryption

This is the point beginners most often get wrong, so let us be blunt about it.

`YWRtaW4=` is not encrypted. It is the word `admin` in base64, and anybody can read it back in one command:

```
echo -n 'YWRtaW4=' | base64 -d # prints: admin
```

You create such a value the same way:

```
echo -n 'admin' | base64 # -n matters: it stops echo adding a newline
# YWRtaW4=
```

WATCH OUT Base64 provides **no security whatsoever**. It exists so binary data can travel safely inside a YAML file — nothing more. A Secret committed to GitHub is a password published to the internet with one extra step, and `YWRtaW4=` is in this repository right now. By default Kubernetes also stores Secrets **unencrypted** in etcd. Never commit real credentials.

TIP The `-n` in `echo -n` is not optional. Without it `echo` appends a newline, that newline is encoded into your password, and MySQL rejects the login with an error that gives no hint about the cause. It is a classic hour-long debugging session.

It only covers half the credentials

Look back at the Deployment's environment block in chapter 19 and you will see this Secret handles `MYSQL_ROOT_PASSWORD` — and that `MYSQL_USER` and `MYSQL_PASSWORD` are **still plain text**, exactly as in [k8s/](#).

NOTE That is a partial migration, and it is worth recognising as such. The pattern is right; it just has not been applied to everything yet. Finishing the job means adding the other two keys to this Secret and switching their `value:` entries to `valueFrom.secretKeyRef` in the Deployment.

Doing it properly

For anything beyond a learning project:

- **Create it imperatively** and never write it to a file at all — `kubectl` does the encoding for you.
- **Sealed Secrets** (Bitnami) — encrypts the Secret so the encrypted form *is* safe to commit; only the controller in your cluster can decrypt it.
- **External Secrets Operator** — leaves the value in AWS Secrets Manager and syncs it in at runtime. This is the natural choice on EKS.

```
kubectl create secret generic mysql-secret \
  --from-literal=MYSQL_ROOT_PASSWORD='a-real-password' # kubectl base64-encodes it for you
```

18. mysql-configmap.yml

From the repo: [eks-manifests/mysql-configmap.yml](#)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-initdb-config
data:
  init.sql: |
    CREATE DATABASE IF NOT EXISTS mydb;
    USE mydb;
    CREATE TABLE messages (id INT AUTO_INCREMENT PRIMARY KEY, message TEXT);
```

ConfigMap is Secret's non-secret twin

A **ConfigMap** works exactly like a Secret — key/value data injected into pods — with two differences: the values are **not** base64-encoded, and it is meant for configuration that is not sensitive. Settings, feature flags, config files, and, as here, a SQL script.

	Secret	ConfigMap
For	Passwords, tokens, keys	Settings, scripts, config files
Values stored as	Base64	Plain text
Can be mounted as files	Yes	Yes
Can be injected as env vars	Yes	Yes

The | is doing real work

```
init.sql: |
  CREATE DATABASE IF NOT EXISTS mydb;
```

The key is `init.sql` and the `|` is YAML's **literal block scalar**. It means "everything indented below this is one multi-line string, and keep the line breaks exactly as written".

Without the `|`, YAML would try to parse those SQL lines as more YAML and fail immediately. With it, the value of `init.sql` is a three-line SQL script — which matters, because when this ConfigMap is mounted as a volume, **each key becomes a file** and its value becomes that file's contents. The pod ends up with a real `init.sql` file on disk.

What the SQL does

```
CREATE DATABASE IF NOT EXISTS mydb; -- the database the app expects
USE mydb; -- switch to it
CREATE TABLE messages (id INT AUTO_INCREMENT PRIMARY KEY, message TEXT); -- the messages
table
```

Same table as `message.sql` in the repo root, which Docker Compose mounted in chapter 6. Three different mechanisms now exist for creating one table:

Where	How the <code>messages</code> table gets created
Docker Compose	<code>message.sql</code> bind-mounted into <code>/docker-entrypoint-initdb.d/</code>
<code>k8s/</code>	<code>app.py</code> runs <code>CREATE TABLE IF NOT EXISTS</code> at startup
<code>eks-manifests/</code>	This ConfigMap, mounted into <code>/docker-entrypoint-initdb.d/</code>

NOTE `CREATE DATABASE` uses `IF NOT EXISTS` but `CREATE TABLE` does not. Re-running this script against a database that already has the table would error. It does not matter in practice, because the MySQL image runs init scripts only once on an empty data directory — but the inconsistency is the kind of thing worth noticing when you read SQL.

19. mysql-deployment.yml and mysql-svc.yml

From the repo: [eks-manifests/mysql-deployment.yml](#)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  labels:
    app: mysql
spec:
```

```

replicas: 1
selector:
  matchLabels:
    app: mysql
template:
  metadata:
    labels:
      app: mysql
  spec:
    containers:
      - name: mysql
        image: mysql:latest
        env:
          - name: MYSQL_ROOT_PASSWORD
            valueFrom:
              secretKeyRef:
                name: mysql-secret
                key: MYSQL_ROOT_PASSWORD
          - name: MYSQL_DATABASE
            value: "mydb"
          - name: MYSQL_USER
            value: "admin"
          - name: MYSQL_PASSWORD
            value: "admin"
        ports:
          - containerPort: 3306
        volumeMounts:
          - name: mysql-initdb
            mountPath: docker-entrypoint-initdb.d
    volumes:
      - name: mysql-initdb
        configMap:
          name: mysql-initdb-config # Config name

```

The skeleton is exactly the Deployment from chapter 11. Two blocks are new, and one of them has a bug.

[valueFrom / secretKeyRef](#) — a password without hard-coding it

```

- name: MYSQL_ROOT_PASSWORD # the variable name inside the container
  valueFrom: # don't take a literal value...
    secretKeyRef: # ...take it from a Secret
      name: mysql-secret # which Secret (chapter 17)
      key: MYSQL_ROOT_PASSWORD # which key inside it

```

Compare with the [k8s/](#) version, which simply said `value: "admin"`. The difference is the whole point of this folder: the Deployment file now names the Secret and the key, never the value. Kubernetes looks it up, base64-decodes it, and sets the variable inside the container.

WATCH OUT If the Secret does not exist, or the key name is misspelled, the pod stays in `CreateContainerConfigError` and never starts. `kubectl describe pod` names the exact key it could not find — always read it before guessing.

Note the two lines immediately below, still plain text:

```
- name: MYSQL_USER
  value: "admin"
- name: MYSQL_PASSWORD
  value: "admin"
```

The partial migration from chapter 17, visible in the file.

Mounting a ConfigMap as a file

```
volumeMounts:
  - name: mysql-initdb
    mountPath: docker-entrypoint-initdb.d
volumes:
  - name: mysql-initdb
    configMap:
      name: mysql-initdb-config
```

The same two-block handshake as chapter 11, but the volume's source is a **ConfigMap** instead of a PVC. Read it as two sentences:

- 1 **volumes**: "This pod has a volume called `mysql-initdb`; its contents come from the ConfigMap `mysql-initdb-config`."
- 2 **volumeMounts**: "Attach it at this path inside the container."

Because the ConfigMap has one key, `init.sql`, the mounted directory contains one file, `init.sql` — and the MySQL image runs every `.sql` file it finds in `/docker-entrypoint-initdb.d/` on first start. That is how the table gets created.

The missing slash

`mountPath: docker-entrypoint-initdb.d` has **no leading slash**, and every other mount path you have seen in this guide does. It is worth knowing exactly what that means, because the answer is more interesting than a simple error.

Kubernetes **accepts** it. The API server only checks that a `mountPath` is non-empty, unique within the container, and not already claimed as a raw block device — it does not require an absolute path. So the manifest applies cleanly and the pod starts.

What happens next is up to the container runtime. The OCI runtime specification says a mount destination on Linux *should* be an absolute path, but that it **may** be relative, in which case it must be interpreted as relative to `/`. So `docker-entrypoint-initdb.d` becomes `/docker-entrypoint-initdb.d` — exactly where the MySQL image looks for init scripts, and the seeding works.

WATCH OUT In other words, this line works **by accident**, by relying on behaviour the OCI specification explicitly marks as **deprecated**. It is not portable, it will confuse anyone reading the file, and a future runtime is entitled to stop honouring it. The fix is one character — `mountPath: /docker-entrypoint-initdb.d` — and there is no reason not to make it.

TIP This is a good habit to generalise: **always write mount paths absolutely**. The same applies to `WORKDIR` in a Dockerfile and to any path in a config file. A path that happens to resolve correctly today is not the same as a path that says what it means.

The bigger omission: no data volume

Look at what is **not** in this file. There is no `PersistentVolumeClaim`, and nothing is mounted at `/var/lib/mysql`. The only volume is the ConfigMap holding the seed SQL.

WATCH OUT The database has **no persistent storage**. MySQL writes to the pod's temporary filesystem, so every message is lost the moment the pod restarts — and on EKS, pods are restarted routinely by node upgrades, scaling and spot interruptions. The `k8s/` folder at least attempted persistence with a `hostPath` PV. This folder does not.

On EKS the right fix is not to copy the `hostPath` PV across — `hostPath` is even worse on a multi-node managed cluster. Instead, install the **EBS CSI driver** and let a PVC provision a real network-attached EBS volume:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: gp3 # ask the EBS CSI driver for a gp3 volume
  resources:
    requests:
      storage: 10Gi
```

...then add the matching mount back into the Deployment, alongside the ConfigMap one:

```
volumeMounts:
  - name: mysql-initdb
    mountPath: /docker-entrypoint-initdb.d # note the fixed leading slash
  - name: mysqldata
    mountPath: /var/lib/mysql # the data directory, now persistent
volumes:
  - name: mysql-initdb
    configMap:
      name: mysql-initdb-config
  - name: mysqldata
    persistentVolumeClaim:
      claimName: mysql-pvc
```

KEY IDEA A pod can mount **as many volumes as it likes**, from different sources — a ConfigMap for configuration, a PVC for data, a Secret for certificates. Each gets an entry in `volumes` and a matching entry in `volumeMounts`. Seeing several together is normal.

mysql-svc.yml

From the repo: [eks-manifests/mysql-svc.yml](https://github.com/kubernetes-sigs/eks-manifests/blob/master/manifests/mysql-svc.yml)

```

apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306

```

Byte-for-byte identical to the [k8s/](#) version from chapter 12, and rightly so. The database is internal in both environments, so a default [ClusterIP](#) Service named `mysql` is correct either way — and it keeps `MYSQL_HOST: mysql` working unchanged in the app Deployment.

KEY IDEA That this file needed no changes at all is the real lesson of Part IV. Moving between a self-managed cluster and a managed cloud one changes how traffic gets in and how storage and secrets are **provided** — the internal wiring between your tiers stays exactly the same.

20. two-tier-app-deployment.yml and the LoadBalancer

From the repo: [eks-manifests/two-tier-app-deployment.yml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: two-tier-app
  labels:
    app: two-tier-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: two-tier-app
  template:
    metadata:
      labels:
        app: two-tier-app
    spec:
      containers:
        - name: two-tier-app
          image: nitin1094/flaskapp:latest
          env:
            - name: MYSQL_HOST
              value: mysql # MySQL Service name (resolved via cluster DNS)
            - name: MYSQL_PASSWORD
              value: "admin"
            - name: MYSQL_USER
              value: "root"
            - name: MYSQL_DB
              value: "mydb"
          ports:

```

```
- containerPort: 5000
imagePullPolicy: Always
```

This is the chapter 14 Deployment with **one cosmetic difference**: `value: mysql` is unquoted here and `value: "mysql"` was quoted there. YAML treats both identically — a bare `mysql` is already a string — so nothing changes.

TIP Quote a YAML value when it could be mistaken for something else. `value: yes`, `value: no` and `value: on` are parsed as **booleans**, and `value: 5000` as a number, where a container environment variable must be a string. `mysql` is unambiguous either way, but quoting everything in an `env` block is a habit that saves real debugging time.

Everything else — the label wiring, service discovery through `MYSQL_HOST`, the missing health probes, the third image name — is exactly as discussed in chapter 14.

two-tier-app-svc.yml — the real difference

From the repo: [eks-manifests/two-tier-app-svc.yml](https://github.com/stackrox/eks-manifests/blob/master/two-tier-app-svc.yml)

```
apiVersion: v1
kind: Service
metadata:
  name: two-tier-app-service
spec:
  selector:
    app: two-tier-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

Put it beside the `k8s/` version and the change is small but consequential:

	<code>k8s/two-tier-app-svc.yml</code>	<code>eks-manifests/two-tier-app-svc.yml</code>
<code>type</code>	<code>NodePort</code>	<code>LoadBalancer</code>
<code>nodePort</code>	<code>30004</code>	<i>(absent)</i>
<code>port</code>	<code>80</code>	<code>80</code>
<code>targetPort</code>	<code>5000</code>	<code>5000</code>
You reach it at	<code>http://<node-ip>:30004</code>	<code>http://<elb-dns-name></code>

What type: `LoadBalancer` actually does

On a plain cluster, `LoadBalancer` does nothing useful — the Service is created and its `EXTERNAL-IP` stays `<pending>` forever, because there is no cloud provider to ask.

On EKS there is. The AWS cloud controller sees the Service and **provisions a real AWS load balancer** pointing at your pods, then writes its DNS name back into the Service. Find it with:

```
kubectl get svc two-tier-app-service
# NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
# two-tier-app-service LoadBalancer 10.100.12.34 a1b2c3...us-east-1.elb.amazonaws.com
80:31234/TCP
```

Open that [EXTERNAL-IP](#) hostname in a browser and the app is live, on port 80 — no port number in the URL, and no need to know any node's IP.

NOTE `nodePort` disappears from the YAML but not from reality — see [80:31234/TCP](#) in the output above. A `LoadBalancer` Service is a `NodePort` Service with a cloud load balancer in front, so Kubernetes still allocates a node port; you just never use it directly.

WATCH OUT This costs money. Each `LoadBalancer` Service provisions its own AWS load balancer, billed hourly whether or not anyone visits. Delete it when you are finished — `kubectl delete -f eks-manifests/two-tier-app-svc.yml` — and confirm in the EC2 console that the load balancer is really gone, because a leftover one bills silently for months.

TIP One load balancer per Service adds up fast. With several apps, the usual answer is a single **Ingress** plus the AWS Load Balancer Controller: one ALB routing by hostname and URL path to many Services. This project is small enough not to need it, but it is the standard next step.

Putting it all together on EKS

A user's click, end to end:

- 1 The browser resolves the ELB's DNS name and connects on port **80**.
- 2 The AWS load balancer forwards to a node, and into the `two-tier-app-service` Service.
- 3 The Service forwards to a pod matching `app: two-tier-app`, on **port 5000**.
- 4 Flask renders the page, reading messages from MySQL. It connects using `MYSQL_HOST=mysql`, resolved by cluster DNS.
- 5 The `mysql` Service forwards to the pod matching `app: mysql`, on **port 3306**.
- 6 `mysql` answers, using the root password injected from the **Secret** and the table seeded by the **ConfigMap**.

Which file did what

Step	File
Public address on port 80	<code>two-tier-app-svc.yml</code> (type: <code>LoadBalancer</code>)
Flask pods running	<code>two-tier-app-deployment.yml</code>
Stable internal name for the database	<code>mysql-svc.yml</code>
MySQL pod running	<code>mysql-deployment.yml</code>
Root password kept out of the Deployment	<code>mysql-secrets.yml</code>
The <code>messages</code> table	<code>mysql-configmap.yml</code>
The image all of this runs	<code>Dockerfile</code> from Part II

APPENDICES

Appendices

The two folders side by side, a full field reference, common errors decoded, interview questions, a glossary, and every link.

21. The Two Manifest Folders Side by Side

Everything Parts III and IV covered, in one comparison. Same app, same Kubernetes, two environments.

Concern	k8s/ (kubeadm)	eks-manifests/ (EKS)
Files	7	6
Namespace	default	default
Getting traffic in	NodePort at <node-ip>:30004	LoadBalancer, a real AWS ELB on port 80
Database storage	hostPath PV + PVC, 256 MiB	none — data is lost on restart
Root password	Plain text value: "admin"	secretKeyRef to mysql-secret
Other DB credentials	Plain text	Still plain text
The messages table	Created by app.py at startup	Seeded by mysql-initdb-config ConfigMap
A bare Pod example	two-tier-app-pod.yml	none
MySQL Service	mysql, ClusterIP, 3306	identical
App image	nitin1094/flaskapp:latest	identical

What stays exactly the same

Worth noticing, because it is the real lesson: the **internal wiring never changes**. The `mysql` Service, the `MYSQL_HOST: mysql` environment variable, the `app: mysql` and `app: two-tier-app` labels, `containerPort: 5000`, `targetPort: 5000` — all identical in both folders.

Moving between a cluster you built yourself and a managed cloud one changes how traffic gets **in**, and how storage and secrets are **supplied**. It does not change how your tiers talk to each other.

If you were to improve either folder

- 1 Add the missing leading slash to `mountPath` in `eks-manifests/mysql-deployment.yml` — it works today only by relying on deprecated OCI behaviour.
- 2 Add a PVC and a `/var/lib/mysql` mount to the EKS MySQL Deployment, backed by the EBS CSI driver.
- 3 Add `storageClassName: ""` to `k8s/mysql-pvc.yml` so it reliably binds to `mysql-pv`.
- 4 Pin `mysql:latest` to `mysql:8.0` in both Deployments.
- 5 Move `MYSQL_USER` and `MYSQL_PASSWORD` into the Secret too, and add one for the app tier.
- 6 Add a `readinessProbe` on `/` port 5000 to both app Deployments.
- 7 Reconcile the three image names — `two-tier-flask-app`, `nitin1094/two-tier-flask-app`, `nitin1094/flaskapp` — down to one.
- 8 Convert MySQL to a `StatefulSet` once it has real storage.

22. Field Reference Cheat Sheet

Every field used anywhere in this project, in one place.

Dockerfile instructions

Instruction	Runs at	Used here for
FROM python:3.9-slim	build	The base image — Linux plus Python 3.9
FROM python:3.9 AS builder	build	A named build stage (multi-stage only)
WORKDIR /app	build	The working folder for every later instruction
RUN apt-get ...	build	System packages; index deleted in the same layer
COPY requirements.txt .	build	The dependency list alone, so its layer caches well
RUN pip install ...	build	Python dependencies
COPY . .	build	The application source
COPY --from=builder ...	build	Lifting installed packages out of an earlier stage
CMD ["python", "app.py"]	run	Starting Flask on port 5000

Docker Compose keys

Key	Purpose
services:	One block per container
image:	The image to pull (or run, if built separately)
build:	Build from a Dockerfile instead — absent in this project
container_name:	Fixes the container's name, which is also its hostname on the network
environment:	Environment variables passed into the container
ports:	"host:container" port mapping
volumes:	"host-path:container-path" bind mounts
networks:	Which private networks the container joins
depends_on:	Start order — not readiness
restart:	Restart policy, e.g. always
healthcheck:	A command that decides whether the container is working

Fields common to every manifest

Field	Purpose
apiVersion	The API group and version the object belongs to
kind	The object type
metadata.name	Its name, unique within its kind and namespace
metadata.labels	Key/value stickers used by selectors
spec	The desired state

Deployment fields

Field	Purpose
<code>spec.replicas</code>	How many identical pods to keep running
<code>spec.selector.matchLabels</code>	Which pods this Deployment owns. Immutable after creation.
<code>spec.template.metadata.labels</code>	The labels stamped on each pod — must match the selector
<code>spec.template.spec.containers[].name</code>	The container's name inside the pod
<code>spec.template.spec.containers[].image</code>	The image to pull
<code>spec.template.spec.containers[].imagePullPolicy</code>	Always , IfNotPresent or Never
<code>spec.template.spec.containers[].ports.containerPort</code>	Documents the port the app listens on
<code>spec.template.spec.containers[].env[].value</code>	A literal environment variable
<code>spec.template.spec.containers[].env[].valueFrom.secretKeyRef</code>	An environment variable read from a Secret
<code>spec.template.spec.containers[].volumeMounts</code>	Where to attach a volume inside the container
<code>spec.template.spec.volumes</code>	What the pod's volumes actually are

Service fields

Field	Purpose
<code>spec.type</code>	ClusterIP (default), NodePort or LoadBalancer
<code>spec.selector</code>	Which pods to send traffic to
<code>spec.ports[].port</code>	The port the Service listens on
<code>spec.ports[].targetPort</code>	The port on the pod. Defaults to port if omitted.
<code>spec.ports[].nodePort</code>	The port opened on every node, 30000–32767
<code>spec.ports[].protocol</code>	TCP (default), UDP or SCTP

Storage, Secret and ConfigMap fields

Field	Purpose
<code>PV spec.capacity.storage</code>	How much storage this volume offers
<code>PV spec.accessModes</code>	<code>ReadWriteOnce</code> , <code>ReadOnlyMany</code> or <code>ReadWriteMany</code>
<code>PV spec.persistentVolumeReclaimPolicy</code>	<code>Retain</code> keeps the data when the claim goes; <code>Delete</code> wipes it
<code>PV spec.hostPath.path</code>	A folder on the node — demo-only storage
<code>PVC spec.resources.requests.storage</code>	How much storage the pod wants
<code>PVC spec.storageClassName: ""</code>	Bind to an existing PV; do not dynamically provision
<code>Secret type: Opaque</code>	Arbitrary key/value data
<code>Secret data</code>	Values, base64-encoded
<code>ConfigMap data</code>	Values, plain text; <code>\</code>

Jenkinsfile blocks

Block	Purpose
<code>@Library("Shared") _</code>	Loads a shared library of reusable pipeline functions
<code>pipeline { }</code>	The whole pipeline
<code>agent { label "dev" }</code>	Which build machine runs it
<code>stages { }</code>	The ordered list of stages
<code>stage("X") { steps { } }</code>	One phase of work and its commands
<code>sh "..."</code>	Run a shell command
<code>post { success / failure }</code>	Runs after all stages, conditionally

23. Common Errors and What They Mean

The failures you are most likely to hit, and where the cause is explained.

What you see	What it means	Fix
Seed SQL mounted somewhere unexpected	<code>mountPath: docker-entrypoint-initdb.d</code> has no leading slash, so the runtime resolves it relative to <code>/</code>	Add the <code>/</code> (chapter 19)
Pod <code>Pending</code> , PVC <code>Pending</code>	No PV matched the claim	Apply <code>mysql-pv.yml</code> ; check sizes and access modes agree (chapter 10)
PVC <code>Bound</code> but to <code>pvc-<uuid></code>	A default StorageClass provisioned new storage and ignored <code>mysql-pv</code>	Add <code>storageClassName: ""</code> (chapter 10)
<code>CreateContainerConfigError</code>	A referenced Secret or ConfigMap key does not exist	Apply the Secret first; check key spelling (chapters 17–19)
<code>CrashLoopBackOff</code> on the app pod	It cannot reach MySQL, or MySQL is still initialising	<code>kubectl logs</code> ; check <code>MYSQL_HOST</code> and give MySQL a minute

What you see	What it means	Fix
<code>ImagePullBackOff</code>	The image name or tag is wrong	Check which of the three image names you actually pushed (chapter 14)
Service exists but nothing responds	Selector matches no pod labels	<code>kubectl get endpoints</code> — <code><none></code> confirms it (chapter 9)
NodePort URL times out	The node's firewall or security group blocks the port	Allow inbound TCP 30004 (chapter 15)
<code>EXTERNAL-IP</code> stuck at <code><pending></code>	<code>type: LoadBalancer</code> on a cluster with no cloud provider	Use <code>NodePort</code> off-cloud (chapter 20)
Page loads, messages never save	The <code>messages</code> table does not exist	Check the ConfigMap mounted, or that <code>app.py</code> ran <code>init_db()</code>
<code>Access denied for user</code>	The credentials disagree between the two Deployments	Compare the env blocks (chapter 11)
Password change had no effect	MySQL only initialises on an empty data directory	Delete the PVC (or <code>mysql-data/</code>) and start over
Compose: <code>flask-app</code> is <code>unhealthy</code>	There is no <code>/health</code> route and no <code>curl</code> in the image	Expected — see chapter 6
Compose: mysql will not start	<code>UID:GID</code> expanded to <code>:"</code>	<code>export UID=\$(id -u)</code> <code>GID=\$(id -g)</code> first (chapter 6)
Jenkins: <code>No such DSL method 'clone'</code>	The <code>Shared</code> library is not configured	Configure it, or replace the calls with <code>sh</code> steps (chapter 8)

The commands that diagnose almost everything:

```
kubectl describe pod <pod-name> # the Events list at the bottom names the real cause
kubectl logs <pod-name> # what the application itself printed
kubectl get endpoints # proves whether Services found their pods
kubectl get pvc # Bound or Pending, and which volume it actually got
docker compose ps # container status and health, for the Compose path
```

24. Interview Questions and Answers

Q. Why does the Dockerfile copy requirements.txt and run pip install before copying the rest of the source?

A. Because of Docker's layer cache. Dependencies change rarely; source code changes constantly. Installing dependencies in their own layer first means an ordinary code edit only invalidates the final `COPY .` layer, so the slow `pip install` — which also compiles `mysqlclient` with `gcc` — is skipped entirely.

Q. Why are apt-get update, install and rm -rf /var/lib/apt/lists/* joined into one RUN?

A. Two reasons. First, one `RUN` is one layer, so the package index is deleted before the layer is saved and never enters the image; deleting it in a later `RUN` would leave it baked into the earlier layer. Second, splitting `update` from `install` lets Docker reuse a stale index layer, which causes the classic "404 Not Found" apt failure.

Q. What is a multi-stage build and what does it buy you here?

A. A Dockerfile with more than one `FROM`, where a later stage copies artefacts from an earlier one via `COPY --from=`. Here the builder stage uses the full `python:3.9` image with gcc and the MySQL dev headers to compile `mysqlclient`, then the final `python:3.9-slim` stage copies only the installed `site-packages` and adds the runtime library `libmariadb3`. The compilers never reach production — roughly 450 MB down to 180 MB, and one less tool for an intruder to use.

Q. What is the difference between RUN and CMD?

A. `RUN` executes during `docker build` and its result is baked into the image as a layer. `CMD` executes when a container starts and sets the default command. `RUN pip install` puts libraries in the image; `CMD ["python", "app.py"]` starts Flask.

Q. Why does app.py bind to 0.0.0.0 instead of 127.0.0.1?

A. `127.0.0.1` accepts connections only from inside the container itself, so port mappings and Services would forward traffic that nothing ever answers. `0.0.0.0` means listen on every interface, which is what makes the container reachable from outside.

Q. In docker-compose.yml, how does the Flask container find MySQL?

A. Through `MYSQL_HOST: mysql`, which is the other service's `container_name`. Both containers join the user-defined `twotier` network, and Docker's built-in DNS resolves container names to IPs on that network. It is the same idea as Kubernetes service discovery, one scale smaller.

Q. depends_on is set, so why does Flask still fail on the first start?

A. Because `depends_on` waits for the container to **start**, not for MySQL to be **ready to accept connections**. MySQL takes 30–60 seconds to initialise, so Flask connects, fails, and is restarted by `restart: always` until the database is up. The proper fix is `depends_on` with `condition: service_healthy`, which uses the health check already defined.

Q. How do a Deployment and a Service find their pods?

A. Entirely through labels. The Deployment's `selector.matchLabels` identifies the pods it owns; a Service's `selector` identifies the pods it routes to. Nothing is wired by IP. If a selector does not match, the Service has no endpoints and traffic silently goes nowhere.

Q. What is the difference between a bare Pod and a Deployment?

A. A Pod is a single instance with no self-healing above the container level — delete it and it is gone. A Deployment holds a pod blueprint plus a desired replica count, so it recreates pods that die, reschedules them when a node fails, scales with one command, and supports rolling updates and rollbacks. `two-tier-app-pod.yml` and `two-tier-app-deployment.yml` in this repo show both.

Q. Explain port, targetPort and nodePort in two-tier-app-svc.yml.

A. `nodePort: 30004` is the port opened on every node's IP, reachable from outside the cluster. `port: 80` is the port the Service itself listens on inside the cluster. `targetPort: 5000` is the port on the pod, where Flask actually listens. The path is browser to `<node-ip>:30004`, to Service `:80`, to pod `:5000`.

Q. When would you choose NodePort over LoadBalancer?

A. NodePort when there is no cloud provider to fulfil a LoadBalancer request — a kubeadm, Minikube or kind cluster, where `type: LoadBalancer` would leave `EXTERNAL-IP` stuck at `<pending>` forever. LoadBalancer on a managed cluster like EKS, where the cloud controller provisions a real load balancer with its own DNS name and no port number in the URL.

Q. Why is the MySQL Service ClusterIP while the app Service is not?

A. Because the database should only ever be reached by pods inside the cluster. `ClusterIP` is the default and makes it unreachable from the internet — a real security decision, not an oversight. Only the tier that users must reach is exposed.

Q. How do a PersistentVolume and a PersistentVolumeClaim relate?

A. The PV is the supply — an actual piece of storage with a size and access mode. The PVC is the demand — a pod's request. Kubernetes binds a claim to a volume that satisfies it. The pod references only the claim, which is what lets you swap the underlying storage without editing the Deployment.

Q. The PVC has no storageClassName. Why does that matter?

A. On a cluster with a default StorageClass — Minikube, kind, any managed cluster — Kubernetes dynamically provisions a brand-new volume and ignores `mysql-pv` completely. Everything appears to work, but the data is not where you intended. Setting `storageClassName: ""` explicitly disables dynamic provisioning and forces the bind to the existing PV. Omitting the field is not the same as setting it empty.

Q. Why is hostPath unsuitable for production?

A. It stores data in a folder on one specific node's local disk. If the pod is rescheduled to another node it finds an empty directory and the data appears to have vanished; if the node is terminated, the data is lost. On EKS you would use the EBS CSI driver to provision network-attached volumes instead.

Q. Kubernetes Secrets are only base64-encoded. Is that secure?

A. No. Base64 is encoding for safe transport, not encryption, and anyone who can read the Secret can decode it in one command — `YWRtaW4=` in this repo is just `admin`. By default Secrets are also stored unencrypted in etcd. For production, enable encryption at rest and use Sealed Secrets or an external store such as AWS Secrets Manager.

Q. What is the difference between a Secret and a ConfigMap?

A. Mechanically almost nothing — both hold key/value data and both can be injected as environment variables or mounted as files. The difference is intent and encoding: Secrets hold sensitive values and store them base64-encoded; ConfigMaps hold ordinary configuration in plain text. This repo uses a Secret for the root password and a ConfigMap for the seed SQL.

Q. How does the messages table get created?

A. Three different ways depending on how you deploy. Under Docker Compose, `message.sql` is bind-mounted into `/docker-entrypoint-initdb.d/`, which the MySQL image runs on first start. Under `k8s/`, nothing seeds it — `app.py` runs `CREATE TABLE IF NOT EXISTS` at startup. Under `eks-manifests/`, a ConfigMap containing `init.sql` is mounted into that same directory.

Q. Why is MySQL a Deployment with one replica rather than three?

A. Because its PersistentVolume is `ReadWriteOnce` — mountable by one node at a time — and two `mysqld` processes writing the same data directory would corrupt it. Scaling a database needs replication, which in Kubernetes means a `StatefulSet` with per-pod storage, not a higher `replicas` count. The Flask tier, being stateless, genuinely can be scaled by changing that number.

Q. Why imagePullPolicy: Always?

A. Because the images are tagged `:latest`, which is mutable — the same tag points at different images over time. `Always` forces the node to check the registry instead of reusing a cached copy. With immutable tags such as a Git commit SHA, `IfNotPresent` would be better and faster.

Q. What does @Library("Shared") _ do in the Jenkinsfile?

A. It loads a Jenkins shared library — a separate Git repository of reusable pipeline functions — which is what makes `clone()`, `trivy_fs()` and `docker_push()` callable. The trailing underscore is required Groovy syntax meaning the annotation applies to the whole script. Without that library configured in Jenkins, the pipeline fails immediately with "No such DSL method".

Q. Why is passing "dockerHubCreds" to docker_push better than putting the password in the Jenkinsfile?

A. It is a credential **ID**, not the secret itself. Jenkins stores the real username and token in its credential store, injects them at run time, and masks them in the build log. The secret never appears in the repository, and rotating it needs no code change.

Q. What would you change to deploy to Kubernetes from this pipeline instead of Compose?

A. Replace the `docker compose up` in the Deploy stage with `kubectl apply -f k8s/` (or `eks-manifests/`), give the agent a kubeconfig with credentials for the cluster, and — since the manifests use `:latest` — add `kubectl rollout restart deployment/two-tier-app` so the new image is actually pulled. Better still, tag images immutably and update the manifest with the new tag.

25. Glossary

Term	Meaning
Base image	The image a Dockerfile builds on top of, named in <code>FROM</code>
Base64	An encoding that represents binary data as text. Not encryption.
Bind mount	A folder on the host mapped into a container, <code>host:container</code>
Build context	The folder sent to the Docker engine by <code>docker build</code> , filtered by <code>.dockerignore</code>
ClusterIP	The default Service type — reachable only from inside the cluster
ConfigMap	An object holding non-secret configuration, injectable as env vars or files
Container	A running instance of an image
Credential ID	A reference to a secret stored in Jenkins, used instead of the secret itself
Declarative	Describing the desired result rather than the steps to reach it
Deployment	A controller that keeps N identical pods alive and rolls out new versions
EBS CSI driver	The EKS add-on that provisions AWS EBS volumes for PVCs
EKS	Amazon Elastic Kubernetes Service — managed Kubernetes on AWS
ELB	Elastic Load Balancer — what <code>type: LoadBalancer</code> provisions on AWS
Endpoint	The pod IP and port a Service currently forwards to
Health check	A command or request that decides whether a container is actually working
hostPath	A volume backed by a folder on one node's own disk
Image	A read-only package containing an application and everything it needs
kubeadm	The tool for building a self-managed Kubernetes cluster by hand
Label	A key/value sticker on an object, matched by selectors
Layer	A filesystem snapshot created by one Dockerfile instruction; the unit of Docker's cache
LoadBalancer	A Service type that asks the cloud provider for a real load balancer
Manifest	A YAML file describing a Kubernetes object
Multi-stage build	A Dockerfile with several <code>FROM</code> stages, keeping build tools out of the final image
Namespace	A folder inside the cluster grouping related objects; this project uses <code>default</code>
NodePort	A Service type that opens a port <code>30000–32767</code> on every node
Pod	The smallest deployable unit — one or more containers sharing a network address
PV	PersistentVolume — storage that exists independently of any pod
PVC	PersistentVolumeClaim — a request for storage, bound to a PV

Term	Meaning
Readiness probe	A check whose failure removes the pod from Service endpoints
Reclaim policy	What happens to a PV's data when its claim is deleted: Retain or Delete
Registry	A server that stores and serves images, such as Docker Hub or ECR
Replica	One of the identical pods a Deployment maintains
Secret	An object holding sensitive key/value data, injected into containers
Selector	A label query determining which objects a controller or Service acts on
Service	A stable name and virtual IP in front of a changing set of pods
Service discovery	Finding another tier by name rather than by IP address
Shared library	A Git repository of reusable Jenkins pipeline functions
StatefulSet	The workload controller for stateful apps — stable names, storage and startup order
StorageClass	A named kind of storage a PVC can request for dynamic provisioning
Tag	The part of an image name after the colon, identifying a version
Trivy	An open-source scanner for vulnerabilities and leaked secrets
Two-tier architecture	Splitting an app into an application tier and a data tier

26. Links and Repository

The source code

All the code explained in this guide is in the author's GitHub repository. Every file quoted here is copied from it verbatim.

From the repo: github.com/nitin1094/two-tier-flask-app-k8s

What	Where
The repository	github.com/nitin1094/two-tier-flask-app-k8s
Dockerfile	Dockerfile
Multi-stage Dockerfile	Dockerfile-multistage
Compose file	docker-compose.yml
CI/CD pipeline	Jenkinsfile
Self-managed manifests	k8s/
EKS manifests	eks-manifests/
Full deployment walkthrough	README.md

Clone it and read along:

```
git clone https://github.com/nitin1094/two-tier-flask-app-k8s.git  
cd two-tier-flask-app-k8s
```

Official documentation

Topic	Link
Dockerfile reference	docs.docker.com/reference/dockerfile
Dockerfile best practices	docs.docker.com/build/building/best-practices
Multi-stage builds	docs.docker.com/build/building/multi-stage
Docker Compose file reference	docs.docker.com/reference/compose-file
Kubernetes Deployments	kubernetes.io/docs/concepts/workloads/controllers/deployment
Kubernetes Services	kubernetes.io/docs/concepts/services-networking/service
Secrets	kubernetes.io/docs/concepts/configuration/secret
ConfigMaps	kubernetes.io/docs/concepts/configuration/configmap
Persistent Volumes	kubernetes.io/docs/concepts/storage/persistent-volumes
Liveness and readiness probes	kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes

Topic	Link
Amazon EBS CSI driver	docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html
Jenkins declarative pipeline	jenkins.io/doc/book/pipeline/syntax
Jenkins shared libraries	jenkins.io/doc/book/pipeline/shared-libraries
Trivy	trivy.dev

Where to go next

- 1 Run it locally** — `docker compose up -d` and open `http://localhost:5000`. Nothing else needed.
- 2 Deploy it** — follow the step-by-step guide in the repository's [README.md](#), which covers building a kubeadm cluster and an EKS cluster.
- 3 Harden it** — work through the eight improvements listed in chapter 21, starting with giving the EKS MySQL Deployment real storage.
- 4 Automate it** — point the Jenkins pipeline at a real cluster, or manage the manifests declaratively with Argo CD.